

ChirpStack — Operator Manual

What this technology does

ChirpStack is the LoRaWAN Network Server / application-event source. It validates LoRaWAN protocol/session state, handles OTAA, receives gateway uplinks, schedules downlinks, manages gateways/devices/profiles, and publishes application events consumed by Node-RED.

It is **not** the source-evidence authority. Gateway journal/evidence services remain a separate security lane.

Current commissioned topology

```
ULC-01 -> ChirpStack-1
ULC-02 -> ChirpStack-2
both -> same PostgreSQL database through local PgBouncer/HAProxy
both -> Valkey through local HAProxy writable-primary endpoint
both -> cloud MQTT through the commissioned workload route
region -> AS923
host private HTTP -> :18080
public normal path -> HTTPS through Reserved IPv4 / HAProxy
```

Current pinned server build uses ChirpStack 4.19.1. Verify the deployed image digest before an upgrade rather than relying on this version string alone.

Step 1 — Confirm both ChirpStack instances exist

Run on ULC-01 and ULC-02:

```
sudo docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}' | grep -i chirpstack || true
sudo ss -lntp | grep ':18080\b' || true
```

PASS means: one intended ChirpStack instance is running per application node and the commissioned private HTTP listener exists.

If a host has no instance, do not immediately create a second one. Check whether it is an intentional maintenance state and inspect the Compose/service definition first.

Step 2 — Verify HTTP readiness on each node

Run locally on each application node:

```
curl -fsS -o /dev/null -w 'HTTP=%{http_code}\n' http://127.0.0.1:18080/
```

If the commissioned listener is bound only to the VPC address on the current build, use that node's documented 10.104.0.x:18080 address instead.

PASS means: HTTP returns 200.

A healthy UI/API response proves the process is serving; it does not yet prove PostgreSQL, Valkey, MQTT, or LoRaWAN traffic.

Step 3 — Check recent logs without dumping secrets

First discover the exact container name from Step 1, then:

```
sudo docker logs --since=15m --tail=150 <CHIRPSTACK_CONTAINER>
```

Look for repeated PostgreSQL, Valkey, MQTT, region, migration, or panic/restart errors.

Do not paste secret-bearing configuration or tokens into the terminal to troubleshoot a connection error.

Step 4 — Verify PostgreSQL dependency path

The current client path is:

```
ChirpStack -> pgbouncer.internal.lorawan.com:6432
-> local PgBouncer
-> local HAProxy :15432
-> current Patroni primary :5432
```

First prove Patroni/PgBouncer independently using the PostgreSQL and HAProxy/PgBouncer guides. If the database path fails, fix that layer before editing ChirpStack.

Step 5 — Verify Valkey dependency path

The current application endpoint is `valkey.internal.lorawan.com:16379` through local HAProxy. Use [09-valkey-sentinel.md](#) to prove that endpoint reaches the current writable Valkey primary.

If the endpoint reports a replica, fix Sentinel/HAProxy routing. Do not hard-code ChirpStack to the currently elected data node.

Step 6 — Verify MQTT dependency path

Use [04-mosquitto-mqtt.md](#) to prove the cloud broker/listener and ChirpStack MQTT workload identity are healthy.

The normal gateway topic hierarchy remains under `as923/gateway/<gateway-eui>/...`. A broker TCP connection without a valid subscription/ACL is not enough.

Step 7 — Verify AS923 consistency

Inspect the effective ChirpStack configuration without printing secret values. Confirm the active region uses the project's **AS923** region configuration and matches:

- Gateway-01 Concentrator region/channel plan;
- MQTT prefix `as923`;
- device profile;
- EMU-01 firmware region/RX settings.

Do not change only the ChirpStack region to make one join test pass.

Step 8 — Verify the registered gateway

In the ChirpStack UI/API, find Gateway EUI:

```
0016c001f139a1cb
```

Check:

- gateway exists once;
- recent last-seen/event activity appears after a known real gateway message;
- region/profile assignments are correct;
- no stale duplicate registration is being used.

If gateway MQTT events reach the broker but ChirpStack last-seen does not update, inspect ChirpStack MQTT subscription/ACL/region before touching the radio.

Step 9 — Verify OTAA layer by layer

For an approved test device, observe one join attempt and answer these questions in order:

1. Did Gateway-01 receive the JoinRequest?
2. Did ChirpStack receive the gateway event?
3. Does the registered DevEUI/JoinEUI match the device?
4. Does protected root-key material match?
5. Did ChirpStack schedule a JoinAccept/downlink?
6. Did the gateway transmit it?
7. Did the device begin post-join uplinks?

Do not regenerate keys merely because step 1 or step 2 failed.

Step 10 — Verify one real application event

Generate one real uplink from the approved device and confirm:

```
Gateway RF event
-> cloud MQTT
-> ChirpStack gateway event
-> ChirpStack application event
-> Node-RED
-> telemetry database
```

For counted research work, use the research recorder/test procedure rather than an improvised manual observation.

Safe configuration/upgrade procedure

1. confirm both nodes are healthy;
2. back up/export current configuration and relevant database state through the documented backup path;

3. pin the new image/version/digest;
4. review release/migration requirements;
5. update one ChirpStack node first;
6. verify HTTP + DB + Valkey + MQTT + one real uplink;
7. only then update the second node;
8. verify coexistence and public path;
9. retain rollback image/config until acceptance is complete.

Do not run competing schema migrations from two nodes simultaneously unless the selected ChirpStack release explicitly supports that workflow.

Troubleshooting order

Symptom	Start here
Both instances down	host/container/runtime/dependency outage
HTTP up, DB errors	PgBouncer/HAProxy/PostgreSQL
HTTP up, Valkey errors	Valkey/Sentinel/HAProxy
Gateway events at broker but not ChirpStack	MQTT workload identity/subscription/region
JoinRequest seen, no JoinAccept	registration/keys/profile/RX/downlink
ChirpStack app event exists, no telemetry	Node-RED/application path
Telemetry exists, no source evidence	evidence services, not ChirpStack

Completion checklist

- both ChirpStack nodes running;
- both private HTTP checks return 200;
- PostgreSQL path healthy;
- Valkey endpoint points to writable primary;
- MQTT subscription/auth path healthy;
- active region is AS923;
- Gateway EUI 0016c001f139a1cb shows fresh activity;
- one approved OTAA/post-join flow succeeds when that test is required;
- one real application event reaches Node-RED/telemetry.

Detailed runbooks

- [../server/cloud-production/09-chirpstack-cloud-cluster.md](#)
- [../server/cloud-production/12-gateway-and-device-migration.md](#)
- [../server/cloud-production/17-troubleshooting.md](#)

Supporting runbook: 09-chirpstack-cloud-cluster.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

9. Two-Node ChirpStack HA on the Minimum Three-Droplet Cluster

Status: COMPLETE / PASS. ChirpStack 4.19.1 is commissioned on ulc-01 and ulc-02 with the approved AS923 configuration, local PgBouncer/Valkey/MQTT HA dependency paths, database migration ownership, reciprocal single-instance survival, two-node coexistence, and clean rejoin proven. Earlier pre-deployment gates below are retained as chronological commissioning evidence; they are not the current runtime state. See 00-current-server-continuation-checkpoint.md for the newest whole-system status.

9.1 Goal and current architecture

The minimum HA profile runs two interchangeable ChirpStack v4 instances:

```
ulc-01 / 10.104.0.2 -> ChirpStack-1
ulc-02 / 10.104.0.4 -> ChirpStack-2
```

Both instances must use the same application database, Valkey state, region configuration, token/JWT secrets, and integration policy. They connect through local HA components rather than directly to elected database or Valkey nodes.

Current commissioned dependency paths:

```
PostgreSQL
ChirpStack-1 -> pgbouncer.internal.lorawan.com:6432 -> PgBouncer ulc-01
ChirpStack-2 -> pgbouncer.internal.lorawan.com:6432 -> PgBouncer ulc-02
PgBouncer -> local HAProxy :15432 -> current Patroni primary

Valkey
ChirpStack-1 -> valkey.internal.lorawan.com:16379 -> HAProxy ulc-01
ChirpStack-2 -> valkey.internal.lorawan.com:16379 -> HAProxy ulc-02
HAProxy -> current Sentinel-elected Valkey primary

MQTT foundation already proven
client TLS name -> mqtt.internal.lorawan.com
HAProxy frontend -> 10.104.0.2:8883
Mosquitto backends -> 10.104.0.2:8884 and 10.104.0.4:8884
```

The latest controlled Valkey failover elected **ulc-02 (10.104.0.4) as primary**. ulc-01 and ulc-03 are replicas. Keep that Sentinel-elected topology; do not manually fail back before ChirpStack commissioning.

9.2 Preconditions and stop conditions

Already proven and reusable:

- three-node PostgreSQL/Patroni HA and the chirpstack database;
- validated logical database backup boundary;
- local PgBouncer :6432 on ulc-01 and ulc-02 with client TLS and SCRAM;
- local HAProxy PostgreSQL primary routing;
- Valkey TLS-only service on three nodes;
- three TLS Sentinels, quorum 2;
- local writable-primary Valkey HAProxy :16379 on ulc-01 and ulc-02;
- automatic Valkey promotion, data survival, HAProxy convergence, and old-primary replica rejoin;
- Mosquitto TLS backends on ulc-01/02 and broker-backend failover behind ulc-01 HAProxy :8883.

Still required before the **first ChirpStack start**:

1. pin the exact ChirpStack image version and immutable digest;
2. extract/inspect the configuration schema and enabled region-file model from that exact image;
3. verify how that image validates the internal PgBouncer and Valkey CAs;
4. inspect the live Mosquitto authentication/ACL configuration rather than assuming the original design file was deployed;
5. create a least-privilege ChirpStack MQTT workload identity using authentication fields actually supported by the pinned ChirpStack image and live Mosquitto configuration;
6. prove the ChirpStack identity can use only the required gateway-backend/application MQTT topics;
7. close the two-application-node MQTT routing gap. The completed Phase 8B implementation has only the validated 10.104.0.2:8883 HAProxy frontend. ChirpStack-2 must not be declared HA while both instances depend on a single HAProxy process;
8. confirm the legal LoRaWAN region and exact active region file;
9. assign migration owner and rollback point.

Public DNS, Reserved IPv4 failover, and HTTPS :443 are **not prerequisites for the first private ChirpStack canary**. Those belong to Phase 10 after the two private ChirpStack instances are healthy.

Stop here: do not create application secrets, run migrations, or start a ChirpStack container if any item above is unresolved.

9.3 First executable step - read-only activation preflight

Run this before installing or changing ChirpStack. The purpose is to capture the actual dependency state and discover whether any old ChirpStack files, containers, ports, or images already exist.

From ulc-03, inspect ulc-01 and ulc-02 using the existing deployment SSH path. The preflight must record, without printing secrets:

```
host name and IP
OS release
Docker Engine + Compose versions
CPU / RAM / disk availability
existing ChirpStack containers/images/directories
port 8080 availability
PgBouncer service + :6432 listener
PostgreSQL login path readiness
HAProxy :16379 listener
```

```
Valkey TLS/master routing through :16379
Mosquitto service state / :8884 on both broker nodes
HAProxy MQTT :8883 state on ulc-01
live Mosquitto config directives relevant to authentication/ACL/TLS
existing MQTT credential/ACL files by metadata only
active region/topic-prefix evidence already present in repository/host config
```

For the live Mosquitto configuration, capture only directive names and non-secret paths/settings. Do **not** print password-file contents, private keys, or future ChirpStack credentials.

Pass this gate only when the database and Valkey client paths are still healthy, 8080 is free on both application nodes, and the MQTT authentication/routing gap is precisely known. The output from this gate becomes the basis for the next mutation step; do not guess from the original Phase 8 target design.

9.3.1 Observed preflight checkpoint - 2026-08-25

The first Phase 9.3 read-only preflight completed far enough to identify the remaining activation blockers without installing or starting ChirpStack.

Validated on both application nodes:

```
OS: Ubuntu 24.04.4 LTS
CPU: 1 vCPU
RAM: ~1.9 GiB
root disk: ~48 GiB, ~17% used
Docker Engine: 29.7.2
Docker Compose: v5.5.0
existing ChirpStack containers: none
existing ChirpStack images: none
existing /etc/lorawan-cloud/chirpstack: absent
existing /opt/chirpstack: absent
existing /etc/chirpstack: absent
PgBouncer: active + enabled
PgBouncer listener: local VPC :6432
HAProxy PostgreSQL listeners: local VPC :15432/:15433
HAProxy Valkey writable-primary listener: local VPC :16379
Mosquitto: active + enabled
Mosquitto TLS backend: :8884
```

MQTT routing remains asymmetric exactly as expected from Phase 8B:

```
ulc-01 10.104.0.2:8883 = present
ulc-02 10.104.0.4:8883 = not yet evidenced/present in the pasted checkpoint
Mosquitto backend :8884 = present on both nodes
```

The live Mosquitto directive inventory showed TLS server configuration on both nodes with `cafile`, `certfile`, `private keyfile`, `require_certificate false`, and `tls_version tlsv1.3`. No active `password_file`, `acl_file`, authentication plugin, or `allow_anonymous` line appeared in the sanitized directive output. Do not infer the effective anonymous-access policy from absence of a line; prove it explicitly before creating the ChirpStack workload identity.

The preflight also exposed a hard blocker on both application nodes:

```
ulc-01 :8080 = IN USE, wildcard bind
ulc-02 :8080 = IN USE, wildcard bind
```

Because ChirpStack is absent, the process currently owning :8080 is another service and must be identified before choosing the ChirpStack private HTTP port. Do not stop or reconfigure that service yet.

Phase 9.3 is therefore **PARTIAL / BLOCKED FOR DIAGNOSTIC FOLLOW-UP**, not failed.

9.3.2 Phase 9.3A blocker diagnostic - 2026-08-25

The follow-up read-only diagnostic narrowed the remaining blockers further.

Port 8080 on both application nodes is definitely **not** published by Docker. The only running containers are the existing Spilo and etcd containers. An HTTP request to 127.0.0.1:8080/ returned HTTP/1.1 200 OK with Content-Type: application/json on both ulc-01 and ulc-02, so a real host process is serving JSON there. The unprivileged `ss -p` output did not reveal its PID/process identity. Do not stop it; the next check must use privileged socket/process inspection.

MQTT routing is now precise:

```
ulc-01 HAProxy MQTT frontend: 10.104.0.2:8883 = PRESENT
ulc-01 HAProxy backend pool: 10.104.0.2:8884 + 10.104.0.4:8884
ulc-02 HAProxy MQTT frontend: 10.104.0.4:8883 = ABSENT
ulc-02 HAProxy MQTT config references = none
Mosquitto TLS backend :8884 = PRESENT on both nodes
```

Therefore ChirpStack-2 cannot yet have an independently local MQTT HAProxy route. This is a real Phase 9 dependency, not merely missing evidence.

Mosquitto authentication remains unresolved. The active inventory still shows `require_certificate false`, and only example ACL/password files were present by metadata. No active `password_file`, `acl_file`, authentication plugin, or explicit `allow_anonymous` directive was observed. Because the earlier recursive `grep` also included historical/backup files, the next check must inspect only the files actually loaded through `/etc/mosquitto/mosquitto.conf` plus active `conf.d/*.conf` files, then perform a harmless unauthenticated TLS connection test against `:8884` to establish effective behavior.

The database/Valkey dependency listeners remained present on both nodes during this diagnostic: PgBouncer :6432, PostgreSQL-primary HAProxy :15432, and Valkey writable-primary HAProxy :16379.

Phase 9.3 remains **PARTIAL / BLOCKED FOR DIAGNOSTIC FOLLOW-UP**. The next read-only continuation is:

1. identify the privileged PID/process/systemd owner of `:8080` on ulc-01 and ulc-02;
2. inspect only the active Mosquitto configuration files and prove the effective unauthenticated TLS behavior on `:8884` without publishing application data;
3. verify the existing chirpstack database login path and Valkey `role:master` routing from each application-host context;
4. after those gates pass, decide whether ChirpStack keeps `:8080` or uses a different private HTTP port, and commission the missing ulc-02 MQTT HAProxy route plus ChirpStack-specific MQTT identity/ACL;
5. only then continue to section 9.4 image pinning/start preparation.

No ChirpStack image pull, configuration directory, secret generation, migration, service restart, or topology change is authorized at this checkpoint.

9.3.3 Phase 9.3B probe harness correction - 2026-08-25

The first Phase 9.3B continuation did **not** produce evidence of an SSH, MQTT, or TLS service failure. Both observed failures happened before the intended checks could reach the target service.

The `:8080` owner command was launched from the unprivileged `opsadmin` shell on ulc-03 while pointing SSH at `/root/.ssh/cloud-deployment-phase8`. That private key is root-readable only, so OpenSSH reported `Identity file ... not accessible: Permission denied` and then `Permission denied (publickey)` for ulc-01 and ulc-02. This is a local key-permission/harness error on ulc-03, not evidence that either application node rejected the already-proven deployment identity. The corrected owner probe must run from the existing root shell (`sudo bash`) that can read the Phase 8 deployment key.

The unauthenticated MQTT probe also stopped before connecting to Mosquitto. `mosquitto_sub` reported Problem setting TLS options: File not found because it ran as unprivileged `opsadmin` while referencing `/etc/lorawan-pki/mqtt/ca.crt`; that commissioned PKI path is not guaranteed to be readable/traversable by the operator account. Since no MQTT CONNECT/CONNACK exchange was reached, the result says nothing about anonymous authorization. The corrected probe must either run the client under remote `sudo` or first use a root-readable CA path, while still omitting username/password/client certificate and publishing no application data.

The active Mosquitto configuration evidence itself remains valid and identical on both nodes:

```
/etc/mosquitto/mosquitto.conf -> include_dir /etc/mosquitto/conf.d
active conf.d/tls.conf -> listener 8884
cafile /etc/lorawan-pki/mqtt/ca.crt
certfile /etc/lorawan-pki/mqtt/server.crt
private keyfile configured
tls_version tlsv1.3
require_certificate false
no active password_file/acl_file/plugin/allow_anonymous directive observed
```

All PgBouncer :6432, PostgreSQL HAProxy :15432, Valkey HAProxy :16379, and Mosquitto :8884 listeners remained present during the failed probe. Phase 9.3 therefore remains **PARTIAL / BLOCKED FOR CORRECTED READ-ONLY PROOF**, with no infrastructure regression established.

9.3.4 Phase 9.3C corrected proof - 2026-08-25

The corrected privileged owner check finally identified the :8080 listener on both application nodes. On `ulc-01` the listener belongs to `postgres` PID 40033; on `ulc-02` it belongs to `postgres` PID 215126. The HTTP response is JSON and exposes keys including `postgresql`, `databases`, `processes`, `system_stats`, `disk_stats`, and `hostname`. This is consistent with the existing Spilo/Patroni PostgreSQL service stack rather than ChirpStack. Therefore port 8080 is **reserved by the commissioned database HA stack and must not be reused or stopped for ChirpStack**. Phase 9 will assign ChirpStack a different private host port after the pinned image confirms its container listen behavior.

The corrected MQTT client did reach TLS negotiation, but still did not reach MQTT authorization. It connected to `127.0.0.1:8884`, while the commissioned broker certificate identity is `mqtt.internal.lorawan.com`; hostname verification therefore failed before an MQTT CONNACK could be observed. Both nodes returned `mosquitto_sub_exit=14` with host name verification failed. This is expected certificate verification behavior and is not an MQTT-authentication failure.

The final read-only MQTT authorization proof must therefore preserve hostname verification while forcing the service name to the local broker IP, for example by temporarily supplying a process-local resolver mapping or connecting to `mqtt.internal.lorawan.com` after a safe local hosts override in a disposable client context. Do not disable hostname verification. The goal is to observe whether an unauthenticated client receives an accepted CONNACK or an authorization rejection.

At this checkpoint:

```
port 8080 owner: known and reserved by PostgreSQL/Spilo stack on both nodes
ulc-01 MQTT HAProxy :8883: present
ulc-02 MQTT HAProxy :8883: absent
Mosquitto TLS backend :8884: healthy on both nodes
Mosquitto effective anonymous authorization: still unresolved
```

Phase 9.3 remains **PARTIAL**, but only the effective MQTT authorization proof and explicit database/Valkey application-path checks remain before dependency closure. Do not pull or start ChirpStack yet.

9.3.5 Phase 9.3D TLS proof - 2026-08-25

Phase 9.3D proved the broker TLS identity cleanly on both Mosquitto backends. Direct TLS connections to 10.104.0.2:8884 and 10.104.0.4:8884 using SNI/hostname `mqtt.internal.lorawan.com` negotiated TLS 1.3 with TLS_AES_256_GCM_SHA384, validated the internal CA, and returned `Verify return code: 0 (ok)`. The certificate subject is `CN = mqtt.internal.lorawan.com` and its SAN set includes the commissioned service identity.

The `mosquitto_sub` runs returned only `exit=124` from the outer timeout and did not print a broker CONNACK or authorization error. That result is not sufficient to classify anonymous MQTT as allowed or denied: it proves neither authentication success nor rejection. The separate `getent hosts mqtt.internal.lorawan.com` check on `ulc-03` returned no mapping, so the next proof must not depend on host DNS or `edit /etc/hosts`.

The remaining authorization proof will therefore use a direct TLS socket to each broker IP while setting the TLS `server_hostname` to `mqtt.internal.lorawan.com`, then send one minimal MQTT 3.1.1 CONNECT packet with no username, password, or client certificate and inspect only the returned CONNACK code. This is read-only from the application's perspective: it creates no subscription, publishes no data, changes no broker state/configuration, and requires no DNS mutation.

Current Phase 9.3 state:

```
:8080 owner on ulc-01/02 = existing Spilo/PostgreSQL host process; preserve it
Mosquitto TLS identity/CA/hostname verification = PASS on both brokers
ulc-01 local HAProxy MQTT :8883 = present
ulc-02 local HAProxy MQTT :8883 = absent
Mosquitto anonymous authorization = still unclassified; final CONNACK proof required
```

Do not start or pull ChirpStack until the CONNACK proof plus the remaining database/Valkey client-path check are recorded.

9.3.6 Phase 9.3E direct MQTT CONNACK proof - 2026-08-25

The final direct MQTT authorization proof passed on both brokers. A minimal MQTT 3.1.1 CONNECT packet with no username, password, or client certificate was sent over a TLS socket that connected directly to each broker IP while still verifying the server as `mqtt.internal.lorawan.com`.

Observed on both `ulc-01` and `ulc-02`:

```
TLS version: TLSv1.3
cipher: TLS_AES_256_GCM_SHA384
verified hostname: mqtt.internal.lorawan.com
raw CONNACK: 20 02 00 05
CONNACK return code: 5
meaning: Not Authorized
result: ANONYMOUS_MQTT_DENIED
```

This conclusively proves that the commissioned Mosquitto listeners reject anonymous MQTT clients while preserving successful TLS certificate and hostname verification. The result is stronger than the earlier configuration-only inventory because it proves the effective broker behavior on both live nodes.

The remaining Phase 9.3 closure items are now limited to the application dependency paths:

1. prove the existing chirpstack PostgreSQL role can connect through each local PgBouncer :6432 endpoint using `verify-full` TLS without printing its password;
2. prove each local Valkey HAProxy :16379 endpoint routes to the current writable master using the existing protected Valkey credential;
3. keep :8080 reserved for the existing Spilo/PostgreSQL process and choose a different private host port for ChirpStack;

4. retain the known MQTT routing gap: ulc-01 has local HAProxy :8883, ulc-02 does not. That route must be commissioned before the two-node ChirpStack deployment is called HA.

Phase 9.3 is **MQTT/TLS/AUTH COMPLETE; DATABASE/VALKEY APPLICATION-PATH CHECKS STILL REQUIRED**. Do not pull or start ChirpStack until those two read-only checks pass.

9.3.7 Phase 9.3F credential-discovery checkpoint - 2026-08-25

The Phase 9.3F application-path proof confirmed that the required local listeners remain present on both application nodes, but it did **not** reach either authenticated client path because the credential-discovery guesses did not match the actual protected secret locations.

Observed on both ulc-01 and ulc-02:

```
PgBouncer :6432 = present
Valkey HAProxy :16379 = present
pgbouncer.internal.lorawan.com = no host mapping returned in the operator shell
DB_SECRET_STATUS = NOT_FOUND
VALKEY_SECRET_STATUS = NOT_FOUND
```

This is a credential-location/discovery issue, not evidence of PostgreSQL, PgBouncer, Valkey, HAProxy, or authentication failure. No login attempt was made because no secret value was loaded. Do not create replacement credentials merely because the filename guesses missed the commissioned files.

Before retrying the application-path proof, inventory only secret **filenames, directories, ownership, permissions, sizes, and modification times** on ulc-01, ulc-02, and ulc-03. Do not print file contents. Prefer existing commissioned credentials over generating new ones.

Phase 9.3 remains **PARTIAL ONLY FOR APPLICATION-PATH PROOF**. MQTT TLS and anonymous-denial checks are complete. The remaining gate is to locate the existing ChirpStack database and Valkey application credentials safely, then prove:

```
chirpstack -> local PgBouncer :6432 -> current writable PostgreSQL primary
application Valkey credential -> local HAProxy :16379 -> current writable Valkey master
```

Only after those two paths pass should section 9.4 begin.

9.3.8 Phase 9.3G partial secret-inventory checkpoint - 2026-08-25

The first Phase 9.3G secret-location inventory returned valid metadata for ulc-01, then stopped before the PgBouncer reference check, ulc-02, and ulc-03. This was a shell-harness issue, not a host/service failure: the find command was supplied candidate starting directories that do not all exist, and under set -e a non-zero find status terminated the remote block even though stderr was redirected.

Valid ulc-01 evidence captured before that stop includes:

```
/etc/lorawan-cloud exists
/etc/pgbouncer exists
/etc/haproxy/secrets exists
/etc/valkey exists
/etc/pgbouncer/userlist.txt = live protected PgBouncer auth file candidate
/etc/haproxy/secrets/valkey-health.env = dedicated HAProxy health credential only; not an application Valkey credential
/etc/valkey/valkey.conf = active Valkey configuration
```

Do not reuse valkey-health.env for ChirpStack: Phase 8 deliberately created that identity as least-privilege HAProxy health-check access. Likewise, do not assume the PgBouncer SCRAM verifier stored in userlist.txt is the original plaintext ChirpStack password required by an application client.

The corrected inventory must first build a list containing only directories that actually exist, then run `find` over that list. It must continue through ulc-01, ulc-02, and ulc-03 and still print metadata only, never file contents.

Phase 9.3 remains **PARTIAL ONLY FOR APPLICATION-PATH PROOF**. No new credentials are authorized yet.

9.3.9 Phase 9.3H corrected secret inventory - 2026-08-25

The corrected secret-safe inventory completed across ulc-01, ulc-02, and ulc-03 without printing credential values.

PgBouncer evidence is consistent on ulc-01 and ulc-02:

```
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
user=chirpstack
user=fabric_adapter
user=telemetry_reader
user=telemetry_writer
```

The protected PgBouncer file therefore proves that the commissioned chirpstack SCRAM verifier is present, but it is not a plaintext application password and must not be treated as one. Phase 6 records confirm that runtime-role authentication was intentionally performed with the operator entering the ChirpStack password at a hidden `/dev/tty` prompt; the repository does not record a plaintext server-side copy of that password.

Valkey evidence is also now precise. ulc-03 retains the root-only commissioned application credential at:

```
/root/lorawan-secrets/valkey-auth.txt
```

Separate root-only files exist for Sentinel authentication and the HAProxy health identity. Keep those identities separate. In particular, do not use `/root/lorawan-secrets/valkey-haproxy-health.txt` or `/etc/haproxy/secrets/valkey-health.env` as the ChirpStack application credential.

The final Phase 9.3 application-path proof must therefore use two different secret-handling methods:

1. prompt the operator for the already-commissioned ChirpStack PostgreSQL password via `/dev/tty` without echoing it, then test the local PgBouncer path on ulc-01 and ulc-02;
2. read `/root/lorawan-secrets/valkey-auth.txt` only inside the root shell on ulc-03 and pass it transiently to the Valkey client without printing it, then test each local HAProxy `:16379` endpoint.

No replacement PostgreSQL password or Valkey credential is needed at this checkpoint.

9.3.10 Phase 9.3I PostgreSQL application-path proof - 2026-08-25

The final PostgreSQL application-path proof passed through both commissioned PgBouncer endpoints using the existing chirpstack SCRAM credential. The host-side `psql` wrapper on ulc-01/02 was not usable because no versioned PostgreSQL client package is installed there, so the successful proof used the existing `psql` client inside the ulc-03 Spilo container while targeting each application node's private PgBouncer endpoint directly.

Observed results:

```
10.104.0.2:6432 -> chirpstack|chirpstack|false|10.104.0.2
10.104.0.4:6432 -> chirpstack|chirpstack|false|10.104.0.2
```

This proves both local PgBouncer endpoints authenticated the intended application identity, selected the chirpstack database, and converged on the same writable Patroni primary. `pg_is_in_recovery() = false` on the returned server confirms the downstream HAProxy primary route is not sending the application to a replica. The locale warnings emitted by the Spilo client environment are a separate non-blocking hygiene issue.

PostgreSQL application path is therefore **PASS**:

```
ulc-01 PgBouncer :6432 -> local HAProxy :15432 -> Patroni primary 10.104.0.2
ulc-02 PgBouncer :6432 -> local HAProxy :15432 -> Patroni primary 10.104.0.2
```

9.3.11 Phase 9.3J Valkey application-path proof - 2026-08-25

The final Valkey application-path proof passed through both commissioned writable-primary HAProxy endpoints using the existing root-only application credential from `/root/lorawan-secrets/valkey-auth.txt` on ulc-03. The credential was read only inside the privileged shell and was not printed.

Observed results:

```
10.104.0.2:16379 -> routed_role=master -> PASS
10.104.0.4:16379 -> routed_role=master -> PASS
```

This proves both application-node HAProxy endpoints independently select the current Sentinel-elected writable Valkey primary and do not route ChirpStack-class traffic to a replica.

Phase 9.3 is therefore **COMPLETE / PASS**. The pre-deployment boundary now proves:

```
ChirpStack not yet installed or started
PostgreSQL path through both PgBouncer endpoints -> writable Patroni primary: PASS
Valkey path through both :16379 endpoints -> writable master: PASS
Mosquitto TLS 1.3 / CA / hostname verification on both brokers: PASS
anonymous MQTT -> denied on both brokers with CONNACK 0x05: PASS
port 8080 -> occupied by existing Spilo/PostgreSQL service and reserved
ulc-01 MQTT HAProxy :8883 -> present
ulc-02 MQTT HAProxy :8883 -> still absent and remains a Phase 9 dependency before two-node HA can be claimed
```

Proceed to Phase 9.4 image pinning and schema inspection. Do not start ChirpStack yet.

9.4 Pin and inspect the exact ChirpStack image

After the preflight, choose the exact supported ChirpStack v4 image and record both tag and immutable digest. Do not use latest.

Before creating configuration, inspect commands supported by that exact image:

```
docker image inspect <PINNED_CHIRPSTACK_IMAGE>
docker run --rm <PINNED_CHIRPSTACK_IMAGE> --version
docker run --rm <PINNED_CHIRPSTACK_IMAGE> --help
```

If the image supports exporting a configuration template, generate it from the image and save a sanitized copy for review. Do not assume the historical `configfile` subcommand exists until `--help` confirms it.

Record:

```
image reference:
immutable digest:
reported ChirpStack version:
configuration-template command:
database migration behavior:
Redis/Valkey URL/TLS fields:
MQTT authentication/TLS fields:
region directory / include behavior:
runtime UID/GID if non-root:
```

The exact image is authoritative for configuration field names. Do not copy old TOML keys merely because they appear in this manual.

9.4.1 Image pinning checkpoint - 2026-08-25

The first Phase 9.4 inspection pinned the ChirpStack release successfully without starting any persistent ChirpStack container.

Observed image identity:

```
release tag: chirpstack/chirpstack:4.19.1
reported version: chirpstack 4.19.1
platform: linux/amd64
runtime user: nobody:nogroup
entrypoint: /usr/bin/chirpstack
immutable image ID: sha256:9e0105f1dd733d3d3caa77aa7cfdbf817417fab8a093dd89639a2cd899ab9efe
repo digest: chirpstack/chirpstack@sha256:9e0105f1dd733d3d3caa77aa7cfdbf817417fab8a093dd89639a2cd899ab9efe
running ChirpStack containers after inspection: none
```

Deployment must use the immutable digest above rather than the mutable tag alone.

The image CLI proves that configfile exists, but the first invocation returned exit code 2 because ChirpStack requires the global `--config <DIR>` argument even for this subcommand. This is a command-invocation issue, not an image or configuration failure. The corrected inspection must provide a disposable empty configuration directory, for example `--config /tmp/chirpstack-config`, then run `configfile`. The generated template must be inspected before creating production files.

Phase 9.4 is therefore **PARTIAL / IMAGE PIN PASS; CONFIGURATION SCHEMA INSPECTION NEXT**. Do not start ChirpStack yet.

9.4.2 Configuration schema checkpoint - 2026-08-25

The corrected configfile invocation against the exact pinned 4.19.1 digest succeeded using a disposable configuration directory. The generated template was 32,124 bytes / 1,052 lines, and no persistent ChirpStack container or database migration was started.

Authoritative 4.19.1 schema findings:

```
PostgreSQL: [postgresql]
dsn="postgresql://...?...sslmode=..."
ca_cert=""
max_open_connections=10
connection_recycling_method="verified"

Valkey / Redis compatibility: [redis]
servers=["redis://..."]
TLS uses rediss://
username/password may be embedded in the URL
cluster=false for the commissioned non-Redis-Cluster HAProxy endpoint
no Redis-specific ca_cert field in ChirpStack 4.19.1

ChirpStack MQTT integration: [integration.mqtt]
server="tcp://127.0.0.1:1883/"
username=""
password=""
client_id=""
share_name="chirpstack"
ca_cert=""
tls_cert=""
tls_key=""
TLS transport uses the image's documented ssl:// scheme

API listener: [api]
bind="0.0.0.0:8080"
```

```
Regions: [network]
enabled_regions=[...]
each enabled name must match a region configuration [[regions]] name
```

The image filesystem contained no packaged `/etc/chirpstack` files or `AS923/regions` files during the inspection. Therefore the deployment must provide the configuration directory and approved region configuration explicitly; do not assume usable region files are embedded in this image.

The internal ChirpStack API listener is confirmed as container `0.0.0.0:8080`. This does **not** change the host-port collision finding: host `:8080` is already owned by Spilo/PostgreSQL on `ulc-01` and `ulc-02`. The eventual container publication must therefore use a different private host port while keeping container port `8080`.

Phase 9.4 is now **COMPLETE / PASS** for image identity and schema discovery. Before creating production configuration, the next checkpoint must establish the approved AS923 region source/content, choose and prove a free private host port on both application nodes, and resolve the remaining `ulc-02` MQTT HAProxy-route asymmetry. Do not start ChirpStack yet.

9.5 Shared configuration directory

9.5.1 Deployment-slot preflight - 2026-08-25

The read-only Phase 9.5A deployment-slot preflight established the following live state before any ChirpStack directory, image, container, firewall rule, or service was created on the application nodes:

```
ulc-01 10.104.0.2:18080 -> FREE
ulc-02 10.104.0.4:18080 -> FREE
ulc-01 host :8080 -> existing Spilo/PostgreSQL service
ulc-02 host :8080 -> existing Spilo/PostgreSQL service
ulc-01 MQTT HAProxy 10.104.0.2:8883 -> PRESENT
ulc-02 MQTT HAProxy 10.104.0.4:8883 -> ABSENT
/etc/lorawan-cloud/chirpstack -> absent on ulc-01 and ulc-02
persistent ChirpStack containers -> none
pinned ChirpStack image on ulc-01/02 -> absent
existing local AS923 / region TOML source on ulc-03 -> none found
```

Port `18080` is therefore the approved candidate private host publication port for ChirpStack, mapping to container `8080`, subject to a final pre-start listener check. Do not bind host `8080`.

The first controlled mutation after this checkpoint is **not** ChirpStack installation. Restore MQTT routing symmetry first by adding the same TLS-passthrough HAProxy frontend/backend design already validated on `ulc-01` to `ulc-02`, changing only the frontend bind address to `10.104.0.4:8883`. Validate the HAProxy configuration before any reload, then verify the listener and both Mosquitto `:8884` backends after reload.

The region source is a separate checkpoint. No AS923 file was found locally, so do not invent or hand-author a region definition from memory. Obtain and review an authoritative ChirpStack 4.19.1-compatible AS923 region file before creating production configuration.

9.5.2 ulc-02 MQTT HAProxy syntax checkpoint - 2026-08-25

The missing `ulc-02` MQTT HAProxy frontend/backend was added as a controlled Phase 9 change, with a timestamped backup created first:

```
/etc/haproxy/haproxy.cfg.phase9-before-mqtt-20260825-060048
```

The new block is intentionally equivalent to the validated `ulc-01` design except for the local frontend bind address:

```

frontend mqtt_tls
  bind 10.104.0.4:8883
  mode tcp
  option tcplog
  default_backend mqtt_brokers

backend mqtt_brokers
  mode tcp
  option tcp-check

  server mqtt-ulc01 10.104.0.2:8884 check
  server mqtt-ulc02 10.104.0.4:8884 check backup

```

haproxy -c -V -f /etc/haproxy/haproxy.cfg returned Configuration file is valid. HAProxy was deliberately **not reloaded** in that step.

9.5.3 ulc-02 MQTT HAProxy activation checkpoint - 2026-08-25

The controlled reload on ulc-02 passed completely. Runtime evidence:

```

haproxy service -> active / enabled
10.104.0.4:8883 -> LISTEN owned by haproxy
10.104.0.2:8884 -> TCP reachable
10.104.0.4:8884 -> TCP reachable
TLS verification through 10.104.0.4:8883 -> OK
verified peername -> mqtt.internal.lorawan.com
TLS version -> TLSv1.3
unauthenticated MQTT CONNECT -> CONNACK 0x05 Not Authorized
existing :6432 / :15432 / :16379 listeners -> still present

```

This closes the Phase 9 MQTT routing asymmetry. Both application nodes now expose equivalent local TLS-passthrough MQTT frontends on their own private :8883 addresses while retaining both Mosquitto :8884 backends. End-to-end TLS terminates only at Mosquitto and anonymous MQTT remains denied through the new ulc-02 route.

The remaining preconfiguration blocker is the authoritative plain-AS923 region file. The frozen deployment identity remains as923 / plain AS923; do not silently substitute as923_2, as923_3, or as923_4. Obtain the region file from an authoritative ChirpStack source compatible with the pinned 4.19.1 release, review its id, common_name, topic_prefix, channels, RX2 parameters, and MQTT backend fields, then preserve its source and checksum before creating production configuration.

9.5.4 authoritative AS923 source checkpoint - 2026-08-25

The first Phase 9.5C source inspection successfully fetched the ChirpStack 4.19.1 source and located the plain-AS923 region file before the inspection harness stopped at its first identity grep.

Verified evidence:

```

source commit: 1ad3e1177c39cc1c566b879898ccf2b96d231260
exact tag reported by repository: api/go/v4.19.1
region path: chirpstack/configuration/region_as923.toml
region SHA-256: ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654

```

The detached-HEAD message is expected for a shallow exact-tag checkout and is not an error. The script then stopped at STEP 5 because its grep required id, description, and common_name to begin at column 1. The region TOML uses nested/indented fields, so grep returned no match and set -e terminated the harness. This does not invalidate the fetched source revision, path, or hash.

Phase 9.5C is therefore **PARTIAL / SOURCE AND HASH PASS; REGION CONTENT INSPECTION NEXT**. Re-run content inspection with whitespace-tolerant matching and non-fatal grep behavior. Do not create the production region file yet.

9.5.5 authoritative AS923 content checkpoint - 2026-08-25

The corrected content inspection passed and confirms that the exact ChirpStack 4.19.1 source file matches the deployment's frozen plain-AS923 identity.

Verified identity and radio parameters:

```
source commit: 1ad3e1177c39cc1c566b879898ccf2b96d231260
repository tag reported: api/go/v4.19.1
region path: chirpstack/configuration/region_as923.toml
region SHA-256: ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654
id: as923
description: AS923
common_name: AS923
MQTT topic_prefix: as923
MQTT share_name: chirpstack
uplink channels: 923200000 Hz and 923400000 Hz, 125 kHz, SF7-SF12
rx1_delay: 1
rx1_dr_offset: 0
rx2_dr: 2
rx2_frequency: 923200000 Hz
min_dr: 0
max_dr: 5
```

The authoritative file's default MQTT server is `tcp://localhost:1883`; this value is only an upstream template default and must be replaced in the production copy with the commissioned local HAProxy TLS route on each application node. Do not change the region identity, channel plan, or RX parameters unless a separately validated regional requirement is introduced.

The plain-AS923 provenance blocker is now **CLOSED / PASS**. Production state remains unchanged: no region file has yet been installed under `/etc/lorawan-cloud/chirpstack` and no ChirpStack container has been started.

The next controlled step may now create the shared configuration directory skeleton on `ulc-01` and `ulc-02` and install a byte-identical copy of this vetted region file, while still leaving the main ChirpStack config, secrets, image pull, and service start for later checkpoints.

9.5.6 vetted AS923 install checkpoint - 2026-08-25

The corrected transfer procedure installed the vetted region file on both application nodes using an unprivileged temporary `scp` path followed by a separate privileged install. Per-node evidence is complete and valid:

```
ulc-01 installed SHA-256: ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654
ulc-02 installed SHA-256: ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654
installed path: /etc/lorawan-cloud/chirpstack/regions/region_as923.toml
ownership/mode: root:root 0644
id: as923
common_name: AS923
topic_prefix: as923
chirpstack.toml: absent
chirpstack.env: absent
ChirpStack container: absent
```

The pasted execution output ended after the `ulc-02` per-node checks and did not include the script's final cross-node summary block or `PHASE 9.5D-R PASS` line. Because the individually reported hashes are already identical and equal to the vetted source hash, the installation itself is **PER-NODE PASS**; however, run one final compact cross-node verification before advancing so the manual also has a clean completed harness result.

9.5.7 final cross-node AS923 hash verification - 2026-08-25

The compact cross-node verification completed successfully and confirmed that both application nodes retain the exact vetted region file hash:

```
10.104.0.2 = ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654
10.104.0.4 = ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654
```

This closes the AS923 file-install boundary completely. The region file is byte-identical across both nodes and matches the authoritative ChirpStack 4.19.1 source hash already recorded above. No main ChirpStack configuration, secret file, image, or container was introduced by this verification.

The next checkpoint is main configuration construction, but the MQTT workload-authentication boundary must be isolated first. The production gateway path is certificate-oriented, while ChirpStack requires a distinct service identity. Do not place a ChirpStack password/ACL policy directly onto the existing gateway-facing broker listener. First commission a dedicated ChirpStack MQTT backend listener on both brokers and a redundant local HAProxy route, then build `chirpstack.toml` from the verified 4.19.1 schema. Keep shared non-secret TOML byte-identical across both nodes where possible; use node-local values only where they genuinely differ.

9.5.8 dedicated ChirpStack MQTT path preflight - 2026-08-25

The read-only preflight confirmed that the proposed dedicated workload ports and files are currently unused on both application nodes:

```
ulc-01 Mosquitto 8885 -> FREE
ulc-02 Mosquitto 8885 -> FREE
ulc-01 HAProxy 18883 -> FREE
ulc-02 HAProxy 18883 -> FREE
/etc/mosquitto/chirpstack.passwd -> absent on both nodes
/etc/mosquitto/chirpstack.acl -> absent on both nodes
/etc/mosquitto/conf.d/chirpstack.conf -> absent on both nodes
Mosquitto version -> 2.0.18 on both nodes
existing 8883 / 8884 listeners -> unchanged
existing 6432 / 15432 / 16379 listeners -> unchanged
```

This reserves :8885 as the dedicated TLS Mosquitto backend listener for ChirpStack workload authentication and :18883 as the candidate local HAProxy frontend on each application node. The existing gateway-facing MQTT path remains separate on HAProxy :8883 -> Mosquitto :8884.

The next mutation must be limited to Mosquitto. Create one shared ChirpStack service credential, a least-privilege ACL allowing the required `as923/gateway/#` and `application/#` operations, and a TLS listener on :8885; validate configuration before reload/restart. Do not modify HAProxy or create `chirpstack.toml` in the same step.

9.5.9 ChirpStack MQTT auth/ACL configuration checkpoint - 2026-08-25

The Phase 9.5F1 run created the shared root-only ChirpStack MQTT secret on ulc-03 and successfully installed the following on ulc-01 before the pasted output stopped:

```
/etc/mosquitto/chirpstack.passwd -> root:mosquitto 0640, hashed password file
/etc/mosquitto/chirpstack.acl -> root:mosquitto 0640
/etc/mosquitto/conf.d/00-auth-scope.conf -> per_listener_settings true
/etc/mosquitto/conf.d/chirpstack.conf -> dedicated TLS listener 10.104.0.2:8885
existing /etc/mosquitto/conf.d/tls.conf -> explicit allow_anonymous false added for :8884
backup stamp -> 20260825-064756
```

The configured ChirpStack ACL is directional: read gateway event/state topics, write gateway command topics, write application event topics, and read application command topics. The dedicated listener uses the commissioned MQTT CA/server certificate/key, TLS 1.3, `allow_anonymous false`, the ChirpStack password file, and the ChirpStack ACL file.

The disposable validation process on ulc-01 returned exit code 124, which is the expected timeout code when the test broker remains running until intentionally terminated. However, the pasted evidence ended immediately after the config-file load lines and did not include the required socket-open assertions,

MOSQUITTO_CONFIG_VALIDATION=PASS, live :8885 absence check, ulc-02 configuration evidence, or the final Phase 9.5F1 completion line.

The compact completion check then proved the split state precisely. On ulc-01, all four ChirpStack MQTT files exist with the expected metadata, `per_listener_settings` true and `explicit_allow_anonymous` false are present, disposable Mosquitto listeners 127.0.0.1:28884 and 127.0.0.1:28885 opened successfully, DISPOSABLE_VALIDATION=PASS, the live Mosquitto service remained active, and live :8885 remained inactive. On ulc-02, none of the four ChirpStack MQTT files exists and disposable validation was correctly skipped; its existing :8883/:8884 service state remains unchanged.

Therefore Phase 9.5F1 is **PARTIAL / ULC-01 COMPLETE PASS; ULC-02 NOT YET CONFIGURED**. Do not restart Mosquitto yet. The next mutation must target ulc-02 only, reuse the already-created root-only ChirpStack MQTT secret from ulc-03, install the same hashed password/ACL/auth-scope policy with node-local bind 10.104.0.4:8885, and repeat the disposable 28884/28885 validation before any live broker restart.

9.5.10 ulc-02 ChirpStack MQTT configuration checkpoint - 2026-08-25

The ulc-02-only completion step passed. It reused the existing root-only ChirpStack MQTT secret from ulc-03, installed a hashed chirpstack password file plus the same directional ACL policy, enabled per-listener authentication scope, made `allow_anonymous` false explicit on the existing :8884 listener, and created the node-local dedicated ChirpStack listener at 10.104.0.4:8885.

Verified evidence:

```
ulc-02 backup stamp: 20260825-065231
/etc/mosquitto/conf.d/00-auth-scope.conf -> root:root 0644
/etc/mosquitto/conf.d/chirpstack.conf -> root:root 0644
/etc/mosquitto/chirpstack.passwd -> root:mosquitto 0640
/etc/mosquitto/chirpstack.acl -> root:mosquitto 0640
live :8885 before restart -> inactive
disposable 127.0.0.1:28884 -> PASS
disposable 127.0.0.1:28885 -> PASS
DISPOSABLE_VALIDATION -> PASS
live Mosquitto service -> active
live :8885 after validation -> inactive
```

Both broker nodes are now configuration-ready for the dedicated ChirpStack MQTT listener, but neither live broker has yet loaded :8885. Phase 9.5F configuration-file preparation is therefore **COMPLETE / PASS**. Activate one live broker at a time, beginning with ulc-01. After each restart, verify that the existing gateway-facing :8884 listener still serves verified TLS and rejects unauthenticated MQTT, while the new :8885 listener verifies TLS, accepts the commissioned ChirpStack username/password on allowed topics, and rejects both anonymous access and an intentionally denied topic. Do not restart both brokers together.

9.5.11 ulc-01 Mosquitto canary activation checkpoint - 2026-08-25

The first live canary activation on ulc-01 partially passed before the pasted output stopped. The Mosquitto service restarted cleanly, remained enabled/active, and loaded both the existing gateway-facing listener and the new ChirpStack workload listener:

```
service state -> active / enabled
0.0.0.0:8884 -> LISTEN
[::]:8884 -> LISTEN
10.104.0.2:8885 -> LISTEN
recent Mosquitto errors -> none
legacy :8884 TLS Verification -> OK
legacy :8884 verified peername -> mqtt.internal.lorawan.com
legacy :8884 verify return code -> 0 (ok)
```

This proves that the new :8885 listener did not prevent the existing :8884 TLS endpoint from returning after restart. However, the pasted evidence stopped before the dedicated :8885 username/password authentication test, allowed-topic ACL test, anonymous-denial test, forbidden-topic ACL test, and the final proof that ulc-02 remained untouched.

Phase 9.5G1 is therefore **PARTIAL / ULC-01 LIVE LISTENER + LEGACY TLS PASS; WORKLOAD AUTH/ACL PROOF STILL REQUIRED**. Do not restart ulc-01 again. Continue with a read-only/runtime-only policy test against the already-live 10.104.0.2:8885, then confirm ulc-02 remains active on :8884 with its :8885 listener still inactive before advancing.

The continuation test then proved TLS 1.3, successful chirpstack username/password authentication (CONNACK 0x00), successful subscription to the allowed gateway-event hierarchy, and anonymous denial (CONNACK 0x05). The intentionally forbidden subscription returned a successful SUBACK instead of 0x80:

```
authenticated_connack = 20020000
MQTT_AUTHENTICATION = PASS
allowed_suback = 9003000100
ACL_ALLOWED_SUBSCRIBE = PASS
anonymous_connack = 20020005
ANONYMOUS_DENIAL = PASS
denied_suback = 9003000200
```

Do **not** interpret that SUBACK alone as proof that the ACL allowed forbidden message delivery. With Mosquitto, subscription acknowledgement and ACL-enforced message delivery must not be conflated in this test. The prior assertion that a forbidden subscription must return SUBACK failure 0x80 was too strict and produced a test-harness false negative. Keep the ACL unchanged until runtime delivery behavior is tested.

The runtime delivery proof then completed successfully without changing the ACL or restarting Mosquitto again. The live ulc-01 :8885 listener accepted the commissioned chirpstack credential over verified TLS 1.3, allowed the configured gateway-event subscription, allowed application-event publication, denied application-event read by proving that no message was delivered to the subscribed ChirpStack client, and allowed gateway-command publication. The previously successful SUBACK on the write-only application-event topic therefore represented subscription acknowledgement only; it did not bypass delivery authorization.

Verified evidence:

```
tls_version = TLSv1.3
MQTT_AUTHENTICATION = PASS
allowed_gateway_suback = 9003000100
ACL_ALLOWED_GATEWAY_READ = PASS
write_only_topic_suback = 9003000200
allowed_event_publish_puback = 40020003
ACL_ALLOWED_APPLICATION_WRITE = PASS
ACL_DENIED_APPLICATION_READ = PASS
gateway_command_puback = 40020004
ACL_ALLOWED_GATEWAY_COMMAND_WRITE = PASS
ULC01_CHIRPSTACK_MQTT_ACL = PASS
ulc-02 Mosquitto = active
ulc-02 live :8885 = NOT_ACTIVE
```

This closes the ulc-01 live canary as **COMPLETE / PASS**. Do not modify its ACL based on SUBACK behavior. The next live mutation is ulc-02 only: restart Mosquitto there, prove the existing gateway-facing :8884 listener still verifies correctly, and repeat the same :8885 authentication, anonymous-denial, allowed-operation, and delivery-enforcement checks before commissioning the redundant HAProxy :18883 frontend.

9.5.12 ulc-02 Mosquitto live activation checkpoint - 2026-08-25

The ulc-02 live activation has partially passed. Before the restart, the ulc-01 canary remained healthy with both :8884 and 10.104.0.2:8885 listening. Mosquitto was then restarted on ulc-02 only and returned active/enabled

without recent error/fatal/failed log entries.

Verified ulc-02 evidence:

```
service state -> active / enabled
0.0.0.0:8884 -> LISTEN
[::]:8884 -> LISTEN
10.104.0.4:8885 -> LISTEN
recent Mosquitto errors -> none
legacy :8884 TLS Verification -> OK
legacy :8884 verified peername -> mqtt.internal.lorawan.com
legacy :8884 verify return code -> 0 (ok)
ulc-01 canary listeners -> PASS throughout
```

This proves the second broker loaded the dedicated ChirpStack listener without regressing the existing gateway-facing TLS listener. However, the pasted output stopped before the ulc-02 :8885 username/password authentication, anonymous-denial, allowed gateway read, application write/read-denial delivery test, allowed gateway command write, and final two-broker :8885 status summary.

Phase 9.5G2 is now **COMPLETE / PASS**. The continuation proof on ulc-02 verified TLS 1.3 and hostname validation on 10.104.0.4:8885, accepted the commissioned chirpstack credential, rejected anonymous MQTT with CONNACK 0x05, allowed gateway-event reads, allowed application-event writes, blocked application-event read delivery, and allowed gateway-command writes. Final runtime state also confirmed both ulc-01 and ulc-02 Mosquitto services active with their node-local :8885 listeners live.

Verified continuation evidence:

```
ULC02 MQTT_AUTHENTICATION = PASS
ULC02 ANONYMOUS_DENIAL = PASS
ULC02 ACL_ALLOWED_GATEWAY_READ = PASS
ULC02 ACL_ALLOWED_APPLICATION_WRITE = PASS
ULC02 ACL_DENIED_APPLICATION_READ = PASS
ULC02 ACL_ALLOWED_GATEWAY_COMMAND_WRITE = PASS
ULC02_CHIRPSTACK_MQTT_ACL = PASS
ulc-01 Mosquitto = active, :8885 ACTIVE
ulc-02 Mosquitto = active, :8885 ACTIVE
```

Both dedicated ChirpStack Mosquitto backends are therefore commissioned. The next step is HAProxy workload routing: add a local TCP frontend on 10.104.0.2:18883 and 10.104.0.4:18883, each balancing across 10.104.0.2:8885 and 10.104.0.4:8885. Validate syntax first and do not reload HAProxy until both node-local configurations pass haproxy -c.

9.5.13 ChirpStack MQTT HAProxy configuration checkpoint - 2026-08-25

Phase 9.5H1 completed successfully on both application nodes. The dedicated ChirpStack MQTT HAProxy frontend/backend definitions were added to each node's existing /etc/haproxy/haproxy.cfg after confirming that local port 18883 was free and that both commissioned Mosquitto :8885 backends were TCP-reachable.

Verified configuration:

```
ulc-01 frontend chirpstack_mqtt_tls -> bind 10.104.0.2:18883
ulc-02 frontend chirpstack_mqtt_tls -> bind 10.104.0.4:18883
backend chirpstack_mqtt_brokers -> active/backup TCP; do not split a ChirpStack MQTT session set across
independent brokers
backend server chirpstack-mqtt-ulc01 -> 10.104.0.2:8885 check
backend server chirpstack-mqtt-ulc02 -> 10.104.0.4:8885 check backup
ulc-01 haproxy -c -> PASS
ulc-02 haproxy -c -> PASS
ulc-01 live :18883 before reload -> inactive
ulc-02 live :18883 before reload -> inactive
```

Rollback backups created immediately before this mutation:

```
ulc-01 /etc/haproxy/haproxy.cfg.phase9-before-chirpstack-mqtt-20260825-071511
ulc-02 /etc/haproxy/haproxy.cfg.phase9-before-chirpstack-mqtt-20260825-071512
```

Existing PgBouncer/PostgreSQL, Valkey, and gateway-facing MQTT HAProxy listeners remained present during validation. HAProxy was not reloaded in Phase 9.5H1, so the new frontend remained configuration-only at this checkpoint.

Phase 9.5H1 is **COMPLETE / PASS**. Activate HAProxy one node at a time. After each reload, prove the node-local :18883 listener is active and that TLS still terminates at Mosquitto with verified peername `mqtt.internal.lorawan.com`; then prove the commissioned ChirpStack username/password authenticates through the frontend. Do not build `chirpstack.toml` until both redundant :18883 paths pass.

9.5.14 ChirpStack MQTT HAProxy activation checkpoint - 2026-08-25

The first Phase 9.5H2 activation completed successfully on ulc-01. HAProxy configuration validation passed, HAProxy reloaded cleanly, and `10.104.0.2:18883` became active while the existing PgBouncer/PostgreSQL, Valkey, and gateway-facing MQTT HAProxy listeners remained present.

Verified ulc-01 end-to-end evidence:

```
haproxy_state = active
10.104.0.2:18883 = ACTIVE
TLS Verification = OK
verified peername = mqtt.internal.lorawan.com
TLS version = TLSv1.3
authenticated CONNACK = 20020000
HAPROXY_MQTT_AUTHENTICATION = PASS
anonymous CONNACK = 20020005
HAPROXY_ANONYMOUS_DENIAL = PASS
10.104.0.2:8885 backend TCP = PASS
10.104.0.4:8885 backend TCP = PASS
```

The harness then stopped on a shell-only status-print bug: `NAME_CHIRPSTACK_MQTT_HAPROXY` was referenced under `set -u` but was never defined. This failure happened **after** all ulc-01 runtime checks had passed and **before** the script reached the ulc-02 HAProxy reload. Therefore no rollback is required on ulc-01, and ulc-01 must not be reloaded again merely to rerun already-passed checks.

Phase 9.5H2 is currently **PARTIAL / ULC-01 COMPLETE PASS; ULC-02 LISTENER ACTIVATED, END-TO-END PROOF STILL REQUIRED**. The Phase 9.5H2-R continuation first reconfirmed ulc-01 HAProxy active with `10.104.0.2:18883` still listening and completed the final ulc-02 pre-reload syntax check with Configuration file is valid. A subsequent minimal no-heredoc recovery step then reloaded ulc-02 HAProxy successfully (`RELOAD_PASS`) and verified the service remained active/enabled with `10.104.0.4:18883` listening under HAProxy PID 507117.

Do not reload either HAProxy again merely to repeat already-passed checks. The remaining Phase 9.5H2 proof was completed read-only through the already-active ulc-02 frontend. TLS verification through `10.104.0.4:18883` returned Verification: OK, verified peername `mqtt.internal.lorawan.com`, and verify return code 0 (ok). An authenticated chirpstack MQTT subscription through the same HAProxy path reached the broker successfully and timed out only because the test topic had no message, which is the expected idle-subscription outcome for this probe. The final two-node state check showed both HAProxy services active with their node-local :18883 listeners present.

Verified final Phase 9.5H evidence:

```
ulc-01 10.104.0.2:18883 -> ACTIVE
ulc-02 10.104.0.4:18883 -> ACTIVE
ulc-01 HAProxy -> active
```

```
ulc-02 HAProxy -> active
ulc-02 TLS Verification -> OK
ulc-02 verified peername -> mqtt.internal.lorawan.com
ulc-02 MQTT authenticated subscription path -> PASS
```

Phase 9.5H is therefore **COMPLETE / PASS**. The redundant ChirpStack MQTT workload path is commissioned on both application nodes. The next phase may construct the production ChirpStack configuration and protected secret layout using local PgBouncer :6432, local Valkey HAProxy :16379, local ChirpStack MQTT HAProxy :18883, approved AS923, and host publication port 18080 -> container 8080.

After the image schema is known and the routing/region prerequisites above are closed, create the same root-owned layout on ulc-01 and ulc-02:

```
sudo install -d -m 750 /etc/lorawan-cloud/chirpstack
sudo install -d -m 750 /etc/lorawan-cloud/chirpstack/regions
sudo install -m 600 /dev/null /etc/lorawan-cloud/chirpstack/chirpstack.env
```

Keep secrets out of Git. Give the container runtime user only the read permission required for mounted configuration and CA files. The active TOML and region files on both nodes must be byte-identical unless a field is explicitly node-specific, such as an MQTT client ID.

Before each start/restart, record hashes:

```
sudo find /etc/lorawan-cloud/chirpstack -type f -exec sha256sum {} +
```

Never print the environment file to collect evidence.

9.6 PostgreSQL connection

Each ChirpStack instance uses its **local PgBouncer**:

```
ulc-01 container
pgbouncer.internal.lorawan.com -> 10.104.0.2
-> 10.104.0.2:6432

ulc-02 container
pgbouncer.internal.lorawan.com -> 10.104.0.4
-> 10.104.0.4:6432
```

The downstream path is:

```
PgBouncer :6432
-> postgres-ha.internal:15432
-> local HAProxy
-> current Patroni primary :5432
```

The DSN must retain hostname verification for pgbouncer.internal.lorawan.com and use the commissioned chirpstack SCRAM credential from protected secret storage. Mount the commissioned PgBouncer CA and use the exact PostgreSQL CA/config field supported by the pinned ChirpStack image.

Illustrative intent only:

```
postgresql://chirpstack:<SECRET_REFERENCE>@pgbouncer.internal.lorawan.com:6432/chirpstack?sslmode=verify-full
```

Do not start ChirpStack until a TLS login to the same local PgBouncer endpoint succeeds from an equivalent container/network context.

9.7 Valkey connection - use the commissioned identity

The commissioned Valkey certificate identity is exactly:

```
valkey.internal.lorawan.com
```

It is **not** `valkey-ha.internal.<DOMAIN>`.

Map the shared name to the local HAProxy address for each application container:

```
ChirpStack-1: valkey.internal.lorawan.com -> 10.104.0.2
ChirpStack-2: valkey.internal.lorawan.com -> 10.104.0.4
```

Both then connect to:

```
redis://...@valkey.internal.lorawan.com:16379
```

HAProxy passes application TLS through to the currently writable Valkey node. The selected backend presents a certificate valid for `valkey.internal.lorawan.com`, so hostname verification remains stable across Sentinel promotion.

The pinned ChirpStack image must trust the commissioned internal Valkey CA. If its Redis/Valkey configuration supports an explicit CA field, use that. If it relies on the operating-system trust store, mount/import the CA by the method supported by that exact image and prove verification from inside a disposable container first.

Never disable certificate verification to make this work.

Before first ChirpStack start, prove from each application host/container context that its local `:16379` endpoint returns a writable `role:master` and that both endpoints survive the current dynamic topology without any hard-coded Valkey primary IP.

9.8 MQTT dependency closure before ChirpStack start

The completed gateway-facing MQTT infrastructure currently proves:

```
certificate name: mqtt.internal.lorawan.com
HAProxy TLS passthrough: 10.104.0.2:8883 and 10.104.0.4:8883
Mosquitto backends: 10.104.0.2:8884 and 10.104.0.4:8884
broker failure detection/recovery: PASS
anonymous MQTT through either HAProxy route: denied
```

The routing asymmetry is closed. The remaining MQTT dependency is the **ChirpStack workload-authentication boundary**. Keep it separate from the gateway-facing broker listener: use dedicated Mosquitto `:8885` listeners with ChirpStack service authentication/ACLs, then expose those through redundant local HAProxy `:18883` frontends.

Before starting ChirpStack:

1. inspect `/etc/mosquitto/mosquitto.conf` and included files on `ulc-01` and `ulc-02`;
2. record the effective `allow_anonymous`, `require_certificate`, `password-file/plugin`, `ACL-file`, `listener`, `CA/certificate/key`, and `TLS-version` directives without printing secrets;
3. inspect the pinned ChirpStack MQTT fields;
4. select one supported workload-authentication method and create a distinct ChirpStack identity;

5. grant only the gateway-backend and application-integration topics required by the exact active region/integration configuration;
6. prove an allowed publish/subscribe operation and a denied operation;
7. make the same reviewed policy available on both Mosquitto backends;
8. commission a second application-node MQTT route or another equally redundant private route before calling the two-node ChirpStack deployment HA.

Because the live broker certificate verifies as `mqtt.internal.lorawan.com`, any TLS-passthrough client route must use that hostname/SNI unless new certificates are deliberately issued. Do not configure `mqtt-ha.internal.<DOMAIN>` from the old target design; that name has not been proven in the commissioned certificate.

The exact ChirpStack MQTT TOML is intentionally **not** hard-coded here until the image preflight proves whether this version expects username/password, client certificate/key, or other fields.

9.9 Region configuration

Use the exact active region-file model from the pinned ChirpStack image. The current repository test/gateway baseline is intentionally frozen on **plain AS923 with MQTT region prefix as923**, not AS923-3. Phase 9 must preserve that existing end-to-end identity unless a deliberate, separately validated migration changes the gateway, sensors, MQTT topic prefix, ChirpStack region, and device profiles together. Copy the exact region ID and frequency plan from the pinned image and the already-approved gateway evidence; do not infer a sub-band from a display label.

Check:

```
end-device region/frequency variant
RAK5146 hardware region
Gateway OS Concentrator channel plan
Gateway MQTT Forwarder topic prefix
ChirpStack gateway-backend MQTT topic prefix
ChirpStack enabled region ID
region frequencies and data rates
device-profile region / MAC settings
antenna band and gain
local regulatory authorization
```

Stop if any layer differs. Do not compensate for a region mismatch in software without correcting the underlying plan.

9.10 Shared token and application secrets

Both ChirpStack nodes must use the same ChirpStack token/JWT secret and any shared application-encryption secret required by the pinned version. Generate/store them only after the configuration schema is known.

Do not rotate a shared token secret on one node independently. Do not write live values into the Compose file, TOML committed to Git, command history, or deployment logs.

9.10.1 Protected secret inventory checkpoint - 2026-08-25

The ulc-03 control node now holds the complete protected ChirpStack secret set under `/root/lorawan-secrets`. Only file metadata was captured; no secret value was printed.

```
-rw----- root:root 34 /root/lorawan-secrets/chirpstack-db-auth.txt
-rw----- root:root 65 /root/lorawan-secrets/valkey-auth.txt
```

```
-rw----- root:root 65 /root/lorawan-secrets/chirpstack-mqtt-auth.txt
-rw----- root:root 45 /root/lorawan-secrets/chirpstack-api-secret.txt
```

Purpose:

```
chirpstack-db-auth.txt    -> existing ChirpStack PostgreSQL SCRAM password
valkey-auth.txt          -> commissioned Valkey application password
chirpstack-mqtt-auth.txt  -> dedicated ChirpStack Mosquitto workload password
chirpstack-api-secret.txt -> shared ChirpStack API/token secret for both HA instances
```

This checkpoint is **COMPLETE / PASS**. Keep these source values only in protected control-node storage while constructing node-local runtime configuration. Do not copy the raw files into Git, deployment logs, or shell command arguments. The next configuration step must generate the ulc-01 canary configuration first and keep ulc-02 ChirpStack stopped until the canary and database-migration boundary are proven.

9.10.2 ChirpStack 4.19.1 environment-substitution checkpoint - 2026-08-25

Source inspection at commit 1ad3e1177c39cc1c566b879898ccf2b96d231260 proves that chirpstack/src/config.rs performs environment-variable substitution on the concatenated contents of every .toml file in the configuration directory before TOML parsing. The implementation is explicit:

```
for (k, v) in env::vars() {
    content = content.replace(&format!("{}", k), &v);
}
```

Therefore the supported placeholder form is literal \$VARNAME. There is no evidence of an automatic CHIRPSTACK__SECTION__FIELD override convention, and this deployment must not invent one.

The same pinned source confirms the relevant secret-bearing main-config fields: postgresql.dsn, redis.servers, integration.mqtt.password, and api.secret. Because substitution occurs before TOML parsing, environment values inserted inside quoted TOML strings must already be safe for that syntactic context; PostgreSQL and Valkey credentials embedded in URLs must also be URL-encoded correctly rather than copied blindly.

The AS923 region TOML is part of the same concatenated configuration and contains its own gateway-backend MQTT server, username, password, and ca_cert settings. The authoritative radio/channel values remain frozen to the vetted AS923 source, but the runtime copy must deliberately replace only those MQTT connection defaults with the commissioned local HAProxy workload path and \$VARNAME secret placeholder. Record a new runtime-file hash after this controlled patch; do not continue calling the modified runtime copy byte-identical to the pristine upstream region source.

Phase 9.6A2 is **COMPLETE / PASS** for environment-substitution semantics.

9.10.3 Exact canary field inspection and Valkey trust gate - 2026-08-25

The Phase 9.6B source inspection confirmed the exact ChirpStack 4.19.1 fields required for the ulc-01 canary. [postgresql] provides dsn, max_open_connections, ca_cert, and connection_recycling_method; [api] provides bind and secret; [network] provides enabled_regions; [integration.mqtt] provides the MQTT server, username, password, client ID, shared-subscription name, CA path, and TLS client-certificate fields. The AS923 region file separately provides the gateway-backend MQTT topic_prefix, share_name, server, username, password, client_id, and ca_cert fields.

The production canary must use connection_recycling_method = "fast" because PgBouncer is already the external connection pooler. The AS923 radio/channel and network parameters remain unchanged; only its gateway-backend MQTT connection fields may be patched for the commissioned mqtt.internal.lorawan.com:18883 workload path.

The same inspection exposed one remaining stop condition: the ChirpStack 4.19.1 [redis] schema contains servers, cluster, key_prefix, and pool settings but no ca_cert field. Do not invent one and do not disable TLS verification. Before writing the ulc-01 production TOML, prove how the exact pinned image and Redis client obtain the trust anchor for rediss://valkey.internal.lorawan.com:16379 (for example, through the image's system CA bundle or another mechanism supported by the exact runtime). Only after that trust path is proven should the canary files be generated.

Phase 9.6B is **COMPLETE / PASS** for field discovery. ChirpStack remains stopped.

9.10.4 Phase 9.6C Valkey TLS trust preflight - PASS - 2026-08-25

The exact pinned ChirpStack 4.19.1 image (sha256:9e0105f1dd733d3d3caa77aa7cfdbf817417fab8a093dd89639a2cd899ab9efe) is Alpine Linux 3.24 and runs as nobody:nogroup. It contains the standard system CA bundle at /etc/ssl/certs/ca-certificates.crt, the /etc/ssl/cert.pem compatibility symlink, the standard CA directories, and update-ca-certificates.

Pinned source commit 1ad3e1177c39cc1c566b879898ccf2b96d231260 confirms that ChirpStack creates its Valkey/Redis pool directly from redis.servers using deadpool_redis::Config::from_url / from_urls. The configuration schema has no Redis-specific ca_cert field. TLS is selected with the documented rediss:// URL scheme.

The same exact source pins the Rust redis crate with tls-rustls and tokio-rustls-comp, and its dependency graph includes rustls-native-certs. This establishes that the Redis TLS path uses native/system certificate roots. Therefore the production canary must keep certificate verification enabled and extend the container trust roots with the internal LoRaWAN CA rather than using an insecure Redis mode or attempting to invent a nonexistent [redis].ca_cert setting.

Implementation rule for the canary: construct a merged CA bundle from the pinned image's /etc/ssl/certs/ca-certificates.crt plus the required internal CA certificate, store it as a root-owned deployment artifact, and mount that merged file read-only at /etc/ssl/certs/ca-certificates.crt inside ChirpStack. Do not replace the system bundle with only the private CA, because ChirpStack may also require normal public trust roots for unrelated HTTPS/TLS integrations.

Phase 9.6C is **COMPLETE / PASS**. No ChirpStack service or persistent container was started.

The final ulc-01 pre-construction check also proved that /etc/lorawan-pki/pgbouncer/ca.crt, /etc/lorawan-pki/valkey/ca.crt, and /etc/lorawan-pki/mqtt/ca.crt are byte-identical (SHA-256 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb) and expose the same CA identity/fingerprint. The merged ChirpStack trust bundle therefore appends this internal CA exactly once.

Important loader correction: the pinned config.rs uses fs::read_dir(config_dir) and loads .toml files directly from that directory; it does not recursively walk a regions/ subdirectory. Keep the pristine vetted source copy under regions/region_as923.toml as provenance, but install the patched active runtime copy as /etc/lorawan-cloud/chirpstack/region_as923.toml and mount it directly into /etc/chirpstack/region_as923.toml beside chirpstack.toml.

The next gate is Phase 9.6D: construct and inspect the ulc-01 canary runtime artifacts (chirpstack.toml, top-level controlled AS923 runtime TOML, merged CA bundle, protected environment file, and container definition) without starting ChirpStack.

9.10.5 Phase 9.6D ulc-01 pre-construction host checkpoint - 2026-08-25

The corrected root-SSH inspection reached ulc-01 successfully and proved the canary slot is still clean. /etc/lorawan-cloud/chirpstack contains only the already-vetted regions/region_as923.toml; no chirpstack.toml, environment file, merged CA bundle, Compose file, or ChirpStack container is present. Host port 18080 remains free.

The installed AS923 file still has the pristine upstream SHA-256

ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654 and still contains the upstream MQTT defaults `server = "tcp://localhost:1883"`, blank username/password/client ID, `topic_prefix = "as923"`, and `share_name = "chirpstack"`. The runtime copy has therefore not yet been patched.

Commissioned CA files on ulc-01 are now proven at `/etc/lorawan-pki/pgbouncer/ca.crt`, `/etc/lorawan-pki/valkey/ca.crt`, and `/etc/lorawan-pki/mqtt/ca.crt`. Each is 1891 bytes, but equal size alone is not proof that they are byte-identical. Before constructing the merged system trust bundle, compare their SHA-256 hashes. If they are identical, append one copy of the internal CA to the pinned image's system bundle; if they differ, append the distinct required CA certificates exactly once each.

Phase 9.6D pre-construction is **PASS / READY FOR ARTIFACT BUILD**, subject only to the CA-hash comparison above. No live service was changed.

The Step-2-to-Step-3 local diagnostic then passed on ulc-03. All four protected ChirpStack secret files were readable and non-empty, PostgreSQL and Valkey URL-encoding produced non-empty results, and root-owned mode-0600 temporary-file creation succeeded. This isolates the earlier automation stop away from secret readability, URL encoding, and local temporary-file handling. Resume from artifact construction only; do not repeat the already-passed remote safety checks and do not start ChirpStack yet. The Phase 9.6D-R continuation then completed local artifact construction through Step 6: protected environment file creation passed without printing secret values, `chirpstack.toml` generation passed, the pristine AS923 hash remained `ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654`, and the active runtime copy was intentionally patched to `ssl://mqtt.internal.lorawan.com:18883` with the ChirpStack workload username, `$_CHIRPSTACK_MQTT_PASSWORD` placeholder, ulc-01 gateway client ID, and merged CA path. The runtime AS923 copy hash became `b6d91fb07a20d403fa543358ecd2c8afc7c64304c98f1cf11bf81f1b5fce8870`. Compose definition construction also passed. The observed output stops at the Step 7 heading before any evidence of staging-directory creation or file transfer, so Steps 7+ remain unproven and must not be marked complete.

A follow-up Step 7B diagnostic proved remote staging-directory creation on ulc-01 at `/tmp/phase96d-probe-535831` with mode `0700` and ownership `opsadmin:opsadmin`. The diagnostic returned to the ulc-03 shell immediately after that proof, so local probe creation, SCP transfer, remote probe verification, and cleanup were not executed in that run and remain unproven. Treat only remote staging creation as PASS.

Harness correction: the first Step 7C continuation used top-level `exit 1` branches directly in the interactive `opsadmin@ulc-03` login shell. If SCP or remote verification failed, that `exit 1` terminated the user's SSH session to ulc-03. This is a command-harness bug, not evidence of an infrastructure failure. All future continuation blocks that may call `exit` must run inside a subshell / `sudo bash -c` boundary so failure returns control to the login shell instead of closing it.

The safe Step 7C subshell harness was then tested from `ulc-03`. The subshell boundary worked as intended: the command returned `STEP_7C_EXIT_CODE=1` without terminating the SSH login session. The observed failure occurred before SCP or remote-file verification because the local `sudo` credential prompt failed (Sorry, try again). Therefore SCP, remote probe verification, and cleanup remained unproven at that checkpoint; this was a local `sudo`-authentication harness issue only. Future continuation commands must refresh `sudo` once at the start (`sudo -v`) and then run the remaining work inside a protected subshell.

Step 7D then completed the staging transport proof successfully after refreshing the `sudo` credential cache. Remote staging existed with mode `0700` and `opsadmin:opsadmin` ownership, local probe creation passed, SCP to ulc-01 passed, the remote probe was verified, cleanup passed, and the protected wrapper returned `STEP_7D_EXIT_CODE=0` while preserving the ulc-03 login shell. Phase 9.6D Step 7 is therefore **COMPLETE / PASS**.

Continuation correction: the Step 3-6 candidate files were created under a `mktemp -d` directory protected by an `EXIT` cleanup trap. Because the interrupted automation returned to the shell before the real transfer, those temporary files were deleted even though their construction logic passed. Therefore the production continuation must regenerate the candidate `env/TOML/runtime-region/Compose` artifacts before the actual transfer. This is

regeneration of previously proven build logic, not a need to repeat the earlier host-safety or staging diagnostics. ChirpStack must still remain stopped until static validation completes.

The first Phase 9.6D-E real-deployment continuation failed at its local secret preflight with `FAIL: missing /root/lorawan-secrets/chirpstack-db-auth.txt` and returned `PHASE_9_6D_E_EXIT_CODE=1` while preserving the ulc-03 login shell. This is a privilege-context harness bug, not a missing-secret condition: `sudo -v` refreshed the sudo credential cache but the surrounding (...) block still executed as `opsadmin`, which cannot traverse `/root/lorawan-secrets`. Earlier root-context metadata and readability checks already proved all four secret files exist. No remote artifact transfer, image pull, CA build, installation, Compose validation, or ChirpStack start occurred in this failed run. Correct continuations must access root-only local secrets through `sudo -n test / sudo -n cat` (or execute the whole worker shell as root); do not treat unprivileged [`-s /root/...`] as authoritative evidence that a secret is absent.

The corrected Phase 9.6D-F continuation then completed the entire ulc-01 static canary artifact boundary successfully. All four protected source secrets were verified as root-owned mode `0600`; the final no-overwrite check passed with `10.104.0.2:18080` still free; the protected environment file, main ChirpStack TOML, and active AS923 runtime TOML were regenerated without printing secret values; the runtime AS923 hash remained `b6d91fb07a20d403fa543358ecd2c8afc7c64304c98f1cf11bf81f1b5fce8870`; and the real candidate transfer to ulc-01 passed. The exact pinned image `sha256:9e0105f1dd733d3d3caa77aa7cfd8bf817417fab8a093dd89639a2cd899ab9efe` was pulled and re-verified as `linux/amd64` with runtime user `nobody:nogroup`.

The merged CA bundle was built from the pinned image's 119-certificate system bundle plus exactly one copy of the commissioned internal CA, producing 120 certificates total. Installed canary artifacts are root-owned with `chirpstack.env` mode `0600` and the non-secret TOML/Compose/CA files mode `0644`. `docker compose config --quiet` passed, an isolated `--network none` ChirpStack configfile parse passed, and the actual runtime user `uid=65534(nobody) gid=65533(nogroup)` proved it can read the mounted main TOML, runtime AS923 TOML, and merged CA bundle. Final safety checks also passed: the pristine AS923 provenance file remains unchanged, host port `18080` is still free, no persistent chirpstack container exists, and the commissioned local dependency listeners `:6432`, `:15432`, `:16379`, and `:18883` remain active. The wrapper returned `PHASE_9_6D_F_EXIT_CODE=0` while preserving the ulc-03 login shell.

Phase 9.6D is therefore **COMPLETE / PASS** for static artifact construction. ChirpStack has still not been started, `docker compose up` has not been executed, and ulc-02 remains untouched. The next gate is read-only inspection of the exact 4.19.1 startup/database-migration behavior and a fresh backup/rollback checkpoint before allowing the first ulc-01 canary start.

The subsequent Step 7 remote-staging diagnostic proved only SSH reachability from ulc-03 to ulc-01 as `opsadmin` (`uid=1001`, member of `sudo` and `docker`). The diagnostic returned to the ulc-03 prompt immediately after that identity check, so remote `/tmp` staging creation and SCP transfer remain unproven. Do not infer a staging or transfer failure yet; continue with a minimal no-heredoc create/write/copy probe.

The first Phase 9.6D pre-construction probe did not reach ulc-01. It was launched from the unprivileged `opsadmin` shell while referencing `/root/.ssh/cloud-deployment-phase8`, so OpenSSH failed locally with `Identity file ... not accessible: Permission denied` and then `Permission denied (publickey)`. The pasted nested heredoc was also visibly corrupted. Treat this as a control-node command-harness failure only; it provides no evidence about ulc-01 PKI paths, port `18080`, ChirpStack files, or service state. The corrected short-command root-SSH retry then passed and produced the authoritative host facts recorded above.

The Phase 9.6D artifact-build automation has now been executed through its first two gates. Step 1 passed on ulc-03 with the complete protected secret inventory present. Step 2 passed on ulc-01: the pristine AS923 provenance file still matches its authoritative hash, the internal CA hash is unchanged, host port `18080` is still free, the canary artifact paths are still absent, and no persistent ChirpStack container exists. No later Step 3+ output has yet been recorded, so do not mark the artifact build complete and do not assume `chirpstack.toml`, `chirpstack.env`, the merged CA bundle, runtime region copy, Compose file, or pinned image pull has occurred.

Phase 9.6E migration/startup inspection checkpoint - 2026-08-26

The exact pinned ChirpStack 4.19.1 image

(sha256:9e0105f1dd733d3d3caa77aa7cfd817417fab8a093dd89639a2cd899ab9efe) was inspected without starting the service. Its CLI exposed the expected administrative subcommands, and direct inspection of the pinned binary found the startup markers Setting up PostgreSQL connection pool, Applying schema migrations, Setting up Redis client, and Starting ChirpStack LoRaWAN Network Server. Treat first normal startup as a schema-migration boundary; do not allow both application nodes to start concurrently across this boundary.

The current chirpstack database was then inspected read-only from the local Spilo PostgreSQL 18.6 client on ulc-03. The session reached database chirpstack; pg_is_in_recovery() returned true, confirming the local ulc-03 PostgreSQL member is a replica. The database currently has **zero public tables**, public.seaql_migrations is absent, and no migration rows exist because the migration relation itself is absent. The empty public-table inventory is authoritative for this checkpoint. ChirpStack remained stopped throughout the inspection, host port 18080 remained free, and no persistent ChirpStack container appeared. Phase 9.6E is **COMPLETE / PASS**.

Phase 9.6F then created a fresh pre-migration rollback archive from the current Patroni leader, dynamically discovered as ulc-01 (10.104.0.2). Before dumping, ulc-01 proved writable (pg_is_in_recovery() = false), the chirpstack database still had zero public tables, and public.seaql_migrations remained absent. The new custom-format archive at /home/opsadmin/backups/chirpstack-pre-migration/20260826-005101/chirpstack.dump is 37,050 bytes, mode 0600, owned by opsadmin:opsadmin, and passed pg_restore --list; BACKUP-METADATA.txt and SHA256SUMS were also created and verified successfully. The source backup creation step is **PASS**.

The second-node copy to ulc-03 was repaired and fully verified in place without re-running pg_dump. The initial copy files were root-owned only because the local transfer used sudo scp; ownership was corrected to opsadmin:opsadmin while preserving mode 0600. chirpstack.dump and BACKUP-METADATA.txt both passed sha256sum -c, the copied archive passed pg_restore --list, and source/copy hashes matched byte-for-byte: chirpstack.dump = 7c41f9609e4b0d754ab6bbaeab6eee4864365ce8e6bf4156ae95faa6d617d693; metadata = 2f97a7ae55b8ab69c25260966f25f352f9c694c04e9661ba7dba0b56b5683936. Patroni still had exactly one leader on ulc-01, the chirpstack database remained writable there with zero public tables and no public.seaql_migrations, host port 18080 remained free, and no persistent ChirpStack container existed. Phase 9.6F is therefore **COMPLETE / PASS**.

The fresh immediately-pre-migration rollback boundary is complete. The first controlled ulc-01 start was then attempted at 2026-08-26T00:57:31Z with ulc-02 still isolated. The container entered a restart loop and reached restart count 10; every attempt logged Setting up PostgreSQL connection pool followed by Applying schema migrations and then failed immediately with Error occurred while creating a new object: invalid connection string. ChirpStack never reached Redis initialization, the database remained unchanged at zero public tables with no migration relation, private HTTP stayed unavailable, and the safety trap stopped the ulc-01 container. ulc-02 remained untouched. **No database restore is required because no schema mutation occurred.**

The current root-cause hypothesis is the PostgreSQL DSN's sslmode=verify-full: ChirpStack's core [postgresql] client is backed by the Rust PostgreSQL stack whose supported SSL modes are disable, prefer, and require, while certificate trust is configured separately through [postgresql].ca_cert. Do not change the DSN or restart ChirpStack until the exact pinned 4.19.1 configfile output and the installed redacted DSN structure confirm this mismatch. The next gate is read-only DSN/TLS-mode inspection, followed by a single reviewed config correction if confirmed.

The first Phase 9.6G-R read-only diagnostic confirmed the exact pinned 4.19.1 configuration template explicitly lists only disable, prefer, and require for core [postgresql].dsn SSL mode and separately exposes [postgresql].ca_cert. The diagnostic wrapper nevertheless returned exit code 1 at its machine-check substep because the harness searched for over-exact full comment strings whose whitespace/line wrapping did not match the rendered template. This is a diagnostic-grep failure only; the visible template output is authoritative evidence of the three supported modes. ChirpStack remained stopped and port 18080 remained free. Continue directly by

extracting only the installed DSN `sslmode` and validating the redacted DSN structure; do not re-run the template extraction and do not restart ChirpStack yet.

The Phase 9.6G-R2 continuation then confirmed the installed protected DSN currently uses `sslmode=verify-full`, which is outside the exact 4.19.1 core PostgreSQL mode set proven above. ChirpStack remained exited and host port 18080 remained free. R2 returned exit code 1 only because its non-secret TLS inspection used a multiline `awk` predicate that the remote `awk` parser rejected (unexpected newline or end of string). Treat that as a read-only harness failure only. The DSN-mode mismatch is now confirmed at the installed-config level; finish only the remaining non-secret checks (`ca_cert`, `connection_recycling_method`, and pristine database state) before changing the DSN or restarting ChirpStack.

Phase 9.6G-R3 then completed those remaining checks successfully. The installed DSN still reports `sslmode=verify-full`; the non-secret PostgreSQL configuration is exactly `max_open_connections = 10`, `ca_cert = "/etc/ssl/certs/ca-certificates.crt"`, and `connection_recycling_method = "fast"`; machine checks for the CA path and PgBouncer recycling mode passed; `ulc-01` remained the sole Patroni leader (`/leader` HTTP 200); and the writable `chirpstack` database remained pristine at `f|0|ABSENT` (`pg_is_in_recovery()` = `false`, zero public tables, no `public.seaql_migrations`). ChirpStack remained exited and host port 18080 remained free. The root cause is therefore **CONFIRMED**: only the core PostgreSQL DSN `sslmode=verify-full` is incompatible with ChirpStack 4.19.1.

Phase 9.6G-F1 then applied exactly that single-field correction in `/etc/lorawan-cloud/chirpstack/chirpstack.env`: `sslmode=verify-full` changed to `sslmode=require`. Before mutation there was exactly one `verify-full` occurrence and zero `require` occurrences; afterward there were zero `verify-full` occurrences and exactly one `require` occurrence. The protected env file remained `root:root` mode `0600`; its size changed from 473 to 469 bytes, consistent with only the four-character mode shortening, and its SHA-256 changed from `095f6c921b46b9f3f239c1d52e49a2b2c5f2ed12d9e61dbd293dca8912e6d4a6` to `7e2482a28f35a7a477d373b12e4becb24b4a61b760c9b9f161c961f444800f54`. The separate CA path and `connection_recycling_method = "fast"` remained unchanged. `docker compose config --quiet` and the disposable `--network none` ChirpStack configfile parse both passed. The writable database remained `f|0|ABSENT`, the failed canary container remained exited, port 18080 remained free, and `ulc-02` still had no ChirpStack container. Phase 9.6G-F1 is therefore **COMPLETE / PASS**.

Phase 9.6G-F2 then force-recreated only the `ulc-01` ChirpStack container with the corrected env file. Docker replaced container ID `cee85d0b6321c66c44e15750e1a6e3aa2f2362b8e53d483ccee73779ac8ef3e8` with `a4e9744415079c8b7a41db9f49252cfeee60a7c62f1439eba92ea648de7bfead`; the new process stayed running with restart count 0 and HTTP 200 on `10.104.0.2:18080`. Fresh logs contained exactly one PostgreSQL setup marker, exactly one Applying schema migrations marker, then progressed successfully through Redis setup, AS923 region setup, MQTT integration TLS connection (`chirpstack-ulc01-integration`), gateway-backend MQTT TLS connection (`chirpstack-ulc01-gateway`), and both expected MQTT subscriptions. This proves the `sslmode=require` correction resolved the prior DSN parser failure and that PostgreSQL, Redis, AS923, MQTT TLS/authentication, and private HTTP all progressed far beyond the original failure point.

The retry was **not accepted as a canary PASS** because the running process emitted repeated scheduler errors every second for `device-queue`, `multicast-group` queue, and `FUOTA` queries. The safety gate counted critical errors, stopped `ulc-01` ChirpStack, left `ulc-02` untouched, and did not restore PostgreSQL. Post-start PostgreSQL inspection showed the database changed from zero public tables to exactly one public table:

`__diesel_schema_migrations`; no other public application tables were present. This state requires investigation before any restart or restore. Do not assume migration success merely because startup progressed past Applying schema migrations; inspect the migration-table rows, all non-system schemas/search path, extensions/privileges, and the full nested scheduler error chains first. The immediate next gate is **read-only post-migration forensics** with ChirpStack stopped.

Phase 9.6G-F3 completed that forensic gate read-only. `public.__diesel_schema_migrations` exists but contains **zero rows**, and it is still the only non-system table. The `chirpstack` role is `LOGIN`, owns the database, has database `CONNECT/CREATE` plus `public-schema` `USAGE/CREATE`, and resolves `"$user"`, `public` to `public`, so

ownership, privileges, and search path do not explain the failed initialization. The database currently has only the baseline Spilo extensions (pg_stat_kcache, pg_stat_statements, plpgsql, set_user). Both pg_trgm 1.6 and hstore 1.8 are available on ulc-01, ulc-02, and ulc-03 but are installed in none of them. ChirpStack remained exited, port 18080 remained free, and the final database state was f|1|0: writable primary, one public table, zero recorded Diesel migrations.

The current ChirpStack v4 requirements explicitly require installing the PostgreSQL pg_trgm extension in the chirpstack database before running ChirpStack. The Phase 6 permanent database checkpoint created the chirpstack database and role but did not install pg_trgm, so this is a confirmed deployment-prerequisite oversight. Do **not** add hstore merely because it is available; current ChirpStack v4 core requirements name pg_trgm, not hstore. The next mutation boundary is therefore limited to CREATE EXTENSION pg_trgm on the current Patroni primary while ChirpStack remains stopped, followed by verification that the extension object is visible on both replicas through normal physical replication. Do not restart ChirpStack until that extension gate passes.

Phase 9.6G-F4 completed that prerequisite repair successfully. Patroni still had exactly one leader on ulc-01. Before mutation the chirpstack database state was f|1|0|0: writable primary, only public.__diesel_schema_migrations, zero Diesel migration rows, and no installed pg_trgm. pg_trgm default version 1.6 was re-proven available on ulc-01, ulc-02, and ulc-03. CREATE EXTENSION pg_trgm was then executed once on the writable primary only. The primary reported pg_trgm|1.6|public|postgres, and both replicas subsequently reported the same installed extension through physical replication (ulc-01=f|pg_trgm|1.6, ulc-02=t|pg_trgm|1.6, ulc-03=t|pg_trgm|1.6). The Diesel state otherwise remained unchanged at one bookkeeping table and zero applied migrations; ChirpStack remained exited, host port 18080 remained free, and hstore was not installed. Phase 9.6G-F4 is therefore **COMPLETE / PASS**.

Phase 9.6G-F5 then repeated the ulc-01-only migration canary with pg_trgm present and the corrected sslmode=require DSN. The force-recreated container stayed running with restart count 0, returned private HTTP 200, logged exactly one PostgreSQL setup marker, one Applying schema migrations marker, Redis initialization, AS923 region setup, both MQTT TLS connection lines, both scheduler-loop setup lines, and **zero scheduler/FUOTA errors and zero total ERROR lines** during the observation window. PostgreSQL advanced from f|1|0|1 to f|25|30|1: 25 public tables, 30 Diesel migration rows, and pg_trgm 1.6 installed. All seven required core tables (tenant, application, device_profile, device, gateway, device_queue_item, multicast_group) were present, and both replicas reported the identical 25|30|1 schema/extension state while remaining in recovery. Patroni leadership stayed on ulc-01 and ulc-02 still had no ChirpStack container. This is authoritative evidence that the schema migration itself **SUCCEEDED** and replicated correctly; do not restore the pre-migration backup and do not rerun first-schema initialization.

F5 nevertheless returned exit code 1 because two harness counters reported mqtt_integration_connect_markers=0 and mqtt_gateway_connect_markers=0. That decision is inconsistent with the same saved sanitized log printed immediately above it, which visibly contains both expected MQTT connection lines. Treat the final stop as a **false-negative MQTT marker-count harness defect**, not a failed migration or failed MQTT connection. The safety trap stopped ulc-01 after the otherwise healthy run; the migrated database must be preserved and must not be restored or re-initialized.

The first F5R read-only follow-up also returned exit code 1, this time at the first fixed-string grep against the saved log. ChirpStack was still exited and port 18080 was free, and no runtime or database mutation occurred. Because the same saved log had already visibly rendered the MQTT connection lines while both fixed-string counters returned zero, the remaining hypothesis was a **log-formatting issue** rather than an MQTT connectivity failure.

Phase 9.6G-F5R2 then proved that hypothesis conclusively. cat -v exposed ANSI CSI sequences such as ^[[3m, ^[[0m, and ^[[2m around the structured MQTT fields, and the raw byte dump showed the corresponding 1b 5b ... escape bytes embedded between client_id, the =, and each value. The two actual connection records are therefore present in the saved log, but the visible text is not stored as the contiguous byte string client_id=chirpstack-ulc01-*. F5R2 returned exit code 1 only because it attempted another pre-normalization regex counter under set -euo pipefail before reaching its ANSI-stripping step. This is now a **confirmed ANSI-**

formatting harness defect. It does not weaken the F5 migration, MQTT connection, HTTP, scheduler, or schema evidence, and no runtime/database mutation occurred during F5R2.

Phase 9.6G-F5R3 completed that final evidence-cleanup gate successfully. The saved F5 log was normalized by stripping ANSI CSI sequences before any marker checks; the normalized log shrank from 4,765 bytes to 3,424 bytes with zero escape bytes remaining. It then exposed exactly one `chirpstack-ulc01-integration` MQTT connection line, exactly one `chirpstack-ulc01-gateway` connection line, one command-topic subscription, and one gateway-event subscription. Scheduler/FUOTA error count and total ERROR count were both zero. The already-migrated PostgreSQL state remained `f|25|30|1` on the primary and `t|25|30|1` on both replicas, all seven core application tables remained present, and ulc-02 still had no ChirpStack container. No database restore and no ChirpStack start occurred during F5R3.

Phase 9.6G-F5/F5R3 is therefore the definitive **first-migration PASS**. The migration created 25 public tables and recorded 30 Diesel migrations, `pg_trgm 1.6` is present and replicated, both MQTT clients and subscriptions are proven, and the earlier zero marker counts are conclusively explained by ANSI formatting in raw structured logs. Do not rerun first-schema initialization and do not restore the pre-migration backup. The next state-changing gate is an ordinary ulc-01-only post-migration canary start and soak: start the existing migrated configuration, keep ulc-02 absent, verify restart count stays zero, normalized logs stay free of scheduler/FUOTA/general errors, private HTTP remains healthy, PostgreSQL schema counts stay at 25/30, Patroni leadership is stable, and both MQTT client/subscription markers are present after ANSI normalization before considering a second ChirpStack node.

Phase 9.6G-F6 completed that ordinary post-migration canary successfully. The existing ulc-01 container was started without recreation and retained container ID `c820e58d9fe5de1c31373c98f8401112860bcea62dc8fbc71203a29d05f7e885`. Private HTTP returned 200 immediately and throughout three 20-second soak checkpoints. The container stayed running with restart count 0 and exit code 0. ANSI-normalized fresh logs contained exactly one PostgreSQL setup marker, one normal migration-check marker, Redis setup, AS923 region configuration, one `chirpstack-ulc01-integration` MQTT connection, one `chirpstack-ulc01-gateway` MQTT connection, and both expected subscriptions. Scheduler/FUOTA errors, total ERROR lines, and panic lines were all zero. The database remained exactly `f|25|30|1` on ulc-01 and `t|25|30|1` on both PostgreSQL replicas; Patroni leadership remained on ulc-01. ulc-02 still had no ChirpStack container. Phase 9.6G-F6 is therefore **COMPLETE / PASS**, and ulc-01 is now the accepted running ChirpStack canary.

Phase 9.6G-F7 completed the ulc-02 static artifact boundary successfully while leaving the accepted ulc-01 canary untouched and healthy. ulc-02 passed the local dependency preflight with `:6432`, `:16379`, and `:18883` listening and `:18080` free. The pristine AS923 source remained byte-identical at SHA-256 `ecb6db8db68bb195c838be2e58ff328dde35fb8f347cfa08cce0c1687fc16654`. The proven ulc-01 artifact set was copied to ulc-02; the protected env and merged CA remained byte-identical to ulc-01 (`chirpstack.env` SHA-256 `7e2482a28f35a7a477d373b12e4becb24b4a61b760c9b9f161c961f444800f54`, CA SHA-256 `6cac5fe88679fec9caade14218a8479558889d780881c49895a2a32d2b0286ef`). Only the node-local deltas were applied: MQTT client IDs became `chirpstack-ulc02-integration` and `chirpstack-ulc02-gateway`; all three internal dependency host mappings now resolve to `10.104.0.4`; and the private HTTP publication is `10.104.0.4:18080:8080`. Artifact ownership/modes are root-owned with `chirpstack.env` mode `0600` and the non-secret files mode `0644`.

The exact pinned ChirpStack image digest was pulled on ulc-02 and re-verified as `linux|amd64|nobody:nogroup`; `docker compose config --quiet` and the isolated `--network none` ChirpStack configfile parse both passed. The pristine region source remained unchanged after the build. PostgreSQL remained `f|25|30|1`, ulc-01 remained running|0 with HTTP 200, and ulc-02 still had no persistent ChirpStack container with port 18080 free. Phase 9.6G-F7 is therefore **COMPLETE / PASS**.

The next state-changing gate is the **first ulc-02 coexistence start while ulc-01 remains live**. Do not recreate or restart ulc-01. Start only the already-statically-validated ulc-02 configuration, capture and ANSI-normalize fresh ulc-02 logs before marker checks, and require: ulc-02 HTTP healthy with restart count 0; one PostgreSQL setup and migration-check marker without any schema-count change; Redis and AS923 setup; MQTT connections for `chirpstack-ulc02-integration` and `chirpstack-ulc02-gateway` plus both expected shared subscriptions; zero

scheduler/FUOTA/general errors or panics; database state still exactly 25 public tables / 30 Diesel migrations / pg_trgm 1.6 on the primary and both replicas; and the existing ulc-01 canary still running|0 with HTTP 200. If ulc-02 fails any gate, stop only ulc-02 and preserve the database for diagnosis. Do not restore or rerun first-schema initialization.

Phase 9.6G-F8 reached the two-node coexistence state successfully at the transport/runtime level but **did not pass the application-error gate**. ulc-02 started from an absent-container state, resolved all three runtime host mappings to 10.104.0.4, reached private HTTP 200 after three probes, and both ulc-01 and ulc-02 then stayed running|0 with HTTP 200 through three 20-second coexistence checkpoints. ANSI-normalized ulc-02 logs contained exactly one PostgreSQL setup marker, one normal migration-check marker, Redis setup, AS923 setup, one connection for each unique ulc-02 MQTT client ID, both expected subscriptions, and both \$share/chirpstack/... shared-subscription topics. The gate nevertheless found **1 scheduler/FUOTA error line and 2 total ERROR lines** in the ulc-02 log. The safety trap therefore stopped only ulc-02; ulc-01 was not stopped or restarted, and no PostgreSQL restore was attempted.

Treat F8 as a **controlled coexistence failure requiring log forensics**, not as evidence of a routing, MQTT client-ID, HTTP, or migration failure. The F8R read-only follow-up then captured the exact two ulc-02 errors: the FUOTA scheduler reported Error occurred while creating a new object: query_wait_timeout, immediately followed by chirpstack::storage::postgres: PostgreSQL connection error error=connection closed. No device-queue or multicast scheduler errors occurred. ulc-01 produced zero ERROR lines and zero MQTT conflict lines during the same coexistence window; the database remained f|25|30|1; ulc-01 remained running|0 with HTTP 200; and Patroni leadership remained on ulc-01.

The timing initially pointed at the PgBouncer pool budget: ulc-02 started its FUOTA scheduler at 01:59:32.801755Z and emitted query_wait_timeout at 01:59:47.981855Z, about 15.18 seconds later, matching the commissioned query_wait_timeout = 15. Phase 7 also deliberately uses small POC limits (pool_mode = session, default_pool_size = 3, reserve_pool_size = 1, reserve_pool_timeout = 5, max_db_connections = 8) while the current ChirpStack PostgreSQL pool allows max_open_connections = 10. That budget mismatch remains relevant, but it is **not sufficient by itself to prove pool exhaustion**.

The completed F8P-C continuation added decisive PgBouncer-side evidence and exposed a more specific resolver defect that must be investigated before any connection-budget tuning. The live non-secret settings matched on ulc-01 and ulc-02, database definitions had no per-database pool overrides, and PgBouncer's journal showed the ulc-02 client login followed by repeated backend attempts to postgres-ha.internal:15432. Some attempts resolved to 127.0.1.1:15432 and failed immediately; later attempts resolved to the intended local private endpoint 10.104.0.4:15432 and established TLS successfully. At 01:59:47Z, PgBouncer closed the waiting ChirpStack client with query_wait_timeout (age=15s) and logged pooler error: query_wait_timeout. The same checkpoint reconfirmed both ChirpStack nodes are configured with max_open_connections = 10, the database remains f|25|30|1, ulc-01 remains running|0, and ulc-02 remains exited|0.

Phase 7 explicitly defines postgres-ha.internal as a **node-local logical name that must map to the current host's private VPC IP** before PgBouncer connects to local HAProxy :15432; its validation records show this invariant directly (for example postgres-ha.internal -> 10.104.0.8 on ulc-03). Therefore 127.0.1.1 is not an acceptable ulc-02 backend target for this design. The current root-cause boundary is now **backend hostname resolution / address selection first, pool sizing second**. Do not reduce ChirpStack max_open_connections, raise PgBouncer limits, restart ulc-02, or change PostgreSQL yet. The next gate must inspect ulc-01/02 getent results, /etc/hosts, NSS resolver order, and direct reachability for every address returned for postgres-ha.internal; only after eliminating the invalid 127.0.1.1 mapping should the two-node coexistence test be repeated and the pool budget reconsidered if waits persist.

The first F8D resolver-forensics shell block is **invalid / no evidence** because the pasted multi-function script was corrupted in the interactive shell before execution. Function definitions and control-flow delimiters were broken (r1 / r2 became undefined, stray fi, else, and) tokens were executed, and unrelated tail fragments appeared inside earlier commands). The printed PASS labels from that run are therefore not trustworthy because the corresponding checks did not execute correctly. No resolver, PgBouncer, ChirpStack, or PostgreSQL mutation was performed.

Replace that large wrapper with short sequential SSH probes that have no shell functions, loops, heredocs, or nested control flow; capture ulc-01 and ulc-02 resolution evidence independently before any mutation.

The first minimal F8D-R1 retry also produced **no remote resolver evidence**. Its direct SSH probes used `sudo -n` on ulc-03 so they could access `/root/.ssh/cloud-deployment-phase8`, but that shell no longer had a cached sudo credential; each remote probe stopped locally with `sudo: a password is required` before SSH ran. The only successful check was unauthenticated private HTTP to ulc-01, which still returned 200. Treat F8D-R1 as a local privilege-precondition harness miss, not a host or resolver failure. Before repeating the same minimal SSH probes, run `sudo -v` once interactively on ulc-03; then use `sudo -n ssh` so no additional prompts appear inside the diagnostic. Do not change resolver files, PgBouncer, ChirpStack, or PostgreSQL during this retry.

F8D-R2 then produced valid current ulc-02 resolver evidence. `getent ahostsv4 postgres-ha.internal` returned only `10.104.0.4`; `/etc/hosts` contains `127.0.1.1 ulc-02 ulc-02, 10.104.0.4 ulc-02`, and a separate `10.104.0.4 postgres-ha.internal` mapping, so the logical database name is not currently aliased to loopback. NSS order is files dns. HAProxy is listening exactly on `10.104.0.4:15432`; direct TCP to that private endpoint succeeds while `127.0.1.1:15432` is closed. ulc-02 ChirpStack remained exited|0 and ulc-01 private HTTP remained 200. Therefore the earlier PgBouncer journal target `127.0.1.1:15432` is **not reproducible in the current OS/NSS view** and must not be fixed by blindly editing `/etc/hosts`.

F8D-R3 narrowed the resolver boundary further. PgBouncer on ulc-02 is linked with `libcares.so.2`; there are no explicit `dns_max_ttl`, `dns_nxdomain_ttl`, `dns_zone_check_period`, or `resolv_conf` settings in `pgbouncer.ini`; `/etc/hosts` mtime is `2026-08-24 05:48:41 UTC`, while the current PgBouncer process started later at `2026-08-24 05:50:19 UTC`, so the current correct host-file mapping predates this PgBouncer process. The Unix socket `/run/postgresql/.s.PGSQL.6432` exists and host `/usr/bin/psql` is available. The next read-only gate must query PgBouncer's own `SHOW DNS_HOSTS` cache through the local Unix socket and compare that cached address with both `/etc/hosts` and the direct DNS resolver path. Do not restart PgBouncer before this evidence is captured, because a restart would destroy the cache state under investigation. Keep pool tuning and ChirpStack restart on hold.

F8D-R4 reached the intended ulc-02 host successfully and confirmed PgBouncer is running as host user `postgres`, UID 110, while the OS resolver still returns only `10.104.0.4` for `postgres-ha.internal`; ulc-02 ChirpStack remained exited|0 and ulc-01 HTTP remained 200. The `SHOW DNS_HOSTS / SHOW DNS_ZONES` console attempt did not execute because `/usr/bin/psql` is only Ubuntu's `pg_wrapper` and no versioned `postgresql-client-<version>` package is installed, producing `Error: You must install at least one postgresql-client-<version> package`. Treat this as a client-wrapper harness limitation, not PgBouncer console/authentication evidence. Do not install another PostgreSQL client package solely for this diagnostic. The commissioned Spilo image already contains a working PostgreSQL 18 client; the next gate may run a disposable container from the already-present immutable Spilo image with `/run/postgresql` mounted as required by Unix-socket semantics and numeric UID 110, then query PgBouncer's local pgbouncer console.

F8D-R5 then closed the resolver root-cause boundary decisively. The disposable Spilo PostgreSQL client successfully queried the live PgBouncer admin console without installing host packages. `SHOW DNS_HOSTS` returned exactly one cached host entry for `postgres-ha.internal` whose address list is `127.0.1.1:0,10.104.0.4:0`; `SHOW DNS_ZONES` reported zone `internal`. In the same checkpoint, `getent -s files ahostsv4 postgres-ha.internal` returned only `10.104.0.4`, while `resolvectl query postgres-ha.internal` returned both `127.0.1.1` and `10.104.0.4` and explicitly labeled the result as synthetic. PgBouncer's PID stayed 202718, the diagnostic container removed itself, ulc-02 ChirpStack remained exited|0, and ulc-01 HTTP remained 200.

This is the authoritative explanation for F8: PgBouncer's c-ares resolver cache contains two addresses for the backend hostname and PgBouncer round-robins resolved addresses for new server connections. HAProxy is listening only on the node's VPC address, so attempts to `127.0.1.1:15432` fail while `10.104.0.4:15432` succeeds; repeated bad-address attempts consume the same 15-second window that ended in `query_wait_timeout`. This is a resolver-source defect, not a ChirpStack scheduler, MQTT, Patroni, schema, or proven pool-capacity failure. Do not reduce `max_open_connections`, increase PgBouncer/PostgreSQL limits, edit `/etc/hosts`, or replace the backend hostname with the node IP. The hostname must remain `postgres-ha.internal` because PgBouncer backend TLS is `verify-full` and the common logical name is the failover-safe certificate identity across PostgreSQL members.

F8D-R6 then proved the resolver-file topology and reload boundary. On ulc-02, `/etc/resolv.conf` resolves to `/run/systemd/resolve/stub-resolv.conf` and points at the local `systemd-resolved` stub `127.0.0.53`; `/run/systemd/resolve/resolv.conf` instead lists the upstream DNS servers `67.207.67.2` and `67.207.67.3`. `resolvectl query postgres-ha.internal` still returned the synthetic pair `127.0.1.1` plus `10.104.0.4`, while NSS files and the normal dns source each returned the private VPC address only. PgBouncer `SHOW CONFIG` reported defaults `dns_max_ttl=15`, `dns_nxdomain_ttl=15`, `dns_zone_check_period=0`, and an unset `resolv_conf`; importantly, `resolv_conf` is marked `changeable=no`, so adopting an alternate resolver file requires a controlled PgBouncer restart rather than a runtime `SET/reload-only` change. `SHOW DNS_HOSTS` still contained `127.0.1.1:0,10.104.0.4:0`, proving the bad cache remained live. PgBouncer PID stayed `202718`, ulc-02 ChirpStack stayed `exited|0`, and ulc-01 HTTP stayed `200`.

F8D-R7 did **not** prove the upstream resolver path. The disposable Spilo container used for that test already contained `10.104.0.4 postgres-ha.internal` in its own `/etc/hosts`, so its getent `ahostsv4 postgres-ha.internal` success was satisfied locally and did not demonstrate that the DigitalOcean nameservers know the private logical name. The same checkpoint nevertheless strengthened the `systemd-resolved` diagnosis: `resolvectl query postgres-ha.internal` and `resolvectl query ulc-02` both returned the synthetic pair `127.0.1.1` plus `10.104.0.4`, while live PgBouncer `SHOW DNS_HOSTS` still cached exactly `127.0.1.1:0,10.104.0.4:0`. PgBouncer PID remained unchanged, ulc-02 ChirpStack remained `exited|0`, and ulc-01 HTTP remained `200`.

Prefer a routing-layer repair over a resolver-file override unless a later isolated c-ares test proves the override end-to-end. Phase 7's `postgres_primary` HAProxy frontend is pure TCP passthrough and currently binds only `<THIS_HOST_PRIVATE_IP>:15432`; PgBouncer keeps the logical backend hostname `postgres-ha.internal` and uses `server_tls_sslmode = verify-full`, so hostname verification happens against the PostgreSQL server certificate after TCP passes through HAProxy. On ulc-02, adding a second loopback-only bind `127.0.1.1:15432` to that same `postgres_primary` frontend would make **both** addresses currently synthesized and cached for `postgres-ha.internal` reach the identical `patroni_primary` backend, while preserving the existing private `10.104.0.4:15432` listener and the failover-safe TLS identity. This avoids changing `/etc/hosts`, `systemd-resolved`, PgBouncer pool sizing, PostgreSQL, or TLS verification semantics.

Phase 9.6G-F8D-F1A completed the off-path HAProxy loopback candidate successfully on ulc-02. The starting live state was safe: ChirpStack remained `exited|0`, HAProxy was active, frontend `postgres_primary` contained exactly one live bind at `10.104.0.4:15432`, and `127.0.1.1:15432` was free. A temporary candidate at `/tmp/haproxy.f8d-f1a.cfg` was created from the installed configuration and changed by exactly one line: bind `127.0.1.1:15432` was added immediately after the existing private bind in frontend `postgres_primary`. Structural checks found exactly one private bind and exactly one loopback bind in the candidate, `haproxy -c -f /tmp/haproxy.f8d-f1a.cfg` returned `Configuration file is valid`, and the unified diff contained no other configuration change. Final live checks still showed only `10.104.0.4:15432` listening, proving the installed HAProxy configuration was not modified or reloaded. ulc-02 ChirpStack remained `exited|0` and ulc-01 HTTP remained `200`. F1A is therefore **COMPLETE / PASS**.

Phase 9.6G-F8D-F1B then installed the already-validated candidate as the persistent ulc-02 HAProxy configuration **without reloading HAProxy**. The pre-install gate re-proved ulc-02 ChirpStack `exited|0`, HAProxy active, the F1A candidate syntax valid, and exactly one candidate private bind plus one candidate loopback bind. A timestamped rollback directory `/etc/haproxy/rollback-f8d-20260826-031441` was created; the original installed configuration and rollback copy had identical SHA-256 `cea2b56f9bde49320becd71d9e62717dac3c822ed9100464883258d7eb8c76be`, and `cmp` returned `ROLLBACK_COPY=PASS`. The exact F1A candidate was then installed root-owned mode `0644`; `cmp` returned `EXACT_CANDIDATE_INSTALLED=PASS`, `haproxy -c` returned `Configuration file is valid`, and structural checks found exactly one `10.104.0.4:15432` bind and one `127.0.1.1:15432` bind in the installed file. Runtime remained deliberately unchanged: the live listener was still only `10.104.0.4:15432`, `127.0.1.1:15432` was not yet live, ulc-02 ChirpStack remained `exited|0`, and ulc-01 HTTP remained `200`. F1B is therefore **COMPLETE / PASS**. The persistent file is now ahead of runtime by design.

Phase 9.6G-F8D-F1B-R then proved the live HAProxy process model and removed the final reload ambiguity. `systemd` tracks master PID `457026`, launched as `/usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy-master.sock`. The active listener owner PID `507117` is its direct child and runs

as the HAProxy worker with `-sf 506605 -x sockpair@4 -ws ...; pgrep -P 457026` returned exactly 507117. The unit's ExecReload first runs `haproxy -ws -f $CONFIG -c -q $EXTRA_OPTS` and only then sends `USR2` to `$MAINPID`, confirming that `systemctl reload haproxy` performs a syntax-check-before-signal master/worker reload. The installed configuration still passed `haproxy -c -V`, the only live `:15432` listener remained `10.104.0.4:15432` owned by worker 507117, ulc-02 ChirpStack remained `exited|0`, and ulc-01 HTTP remained `200`. F1B-R is therefore **COMPLETE / PASS**.

Phase 9.6G-F8D-F1C completed the live ulc-02 HAProxy loopback canary successfully. The pre-reload guard showed HAProxy active with master PID 457026, worker 507117, PgBouncer PID 202718, ulc-02 ChirpStack `exited|0`, and the installed configuration valid. `systemctl reload haproxy` then kept the master PID unchanged and cleanly replaced worker 507117 with worker 746292; the old worker exited with code `0` and the master logged `Loading success`. The new worker owns exactly one listener at `10.104.0.4:15432` and exactly one at `127.0.1.1:15432`; direct TCP checks reported both endpoints `OPEN`. The independent replica frontend `10.104.0.4:15433` remained live. PgBouncer was not restarted, ulc-02 ChirpStack remained `exited|0`, and ulc-01 ChirpStack remained `HTTP 200`. F1C is therefore **COMPLETE / PASS**.

The role-health warnings immediately after reload are expected and are not HAProxy failures. `patroni_primary` marked ulc-02 and ulc-03 `DOWN` with `HTTP 503` while leaving one active server, which is correct because only the current Patroni leader should pass `/primary`; `patroni_replicas` marked ulc-01 `DOWN` with `HTTP 503` while leaving two active servers, which is correct because the leader should not pass `/replica`. The Valkey primary backend likewise marked the two non-master nodes down while leaving one active master. These logs demonstrate role filtering after the new worker generation loaded; they do not justify rollback.

Phase 9.6G-F8D-F1D captured the post-repair PgBouncer baseline successfully without changing PgBouncer or starting ChirpStack. `SHOW DNS_HOSTS` still returned the exact historical two-address cache for `postgres-ha.internal: 127.0.1.1:0, 10.104.0.4:0`. `SHOW SERVERS` returned zero rows, proving there were no pre-existing PostgreSQL server connections that could hide the behavior of the next fresh connection. `SHOW POOLS` showed all application pools with `cl_active=0`, `cl_waiting=0`, and zero server slots in use; only the local pgbouncer console client was active. PgBouncer journal filtering since the HAProxy repair returned no `postgres-ha.internal`, `connect failed`, `TLS`, `server-login`, or `query_wait_timeout` records. Both HAProxy primary endpoints remained open, PgBouncer PID remained 202718, HAProxy master PID remained 457026, ulc-02 ChirpStack remained `exited|0`, and ulc-01 HTTP remained `200`. F1D is therefore **COMPLETE / PASS**.

The first attempt at that fresh-session gate, Phase 9.6G-F8D-F1E-1, produced **no PgBouncer backend-selection evidence** because the disposable client failed locally while opening its mounted `sslrootcert`. `psql` returned `could not read root certificate file "/tmp/pgbouncer-ca.crt" and exited 2` before completing TLS. PgBouncer therefore logged only a client-side TLS handshake EOF; `SHOW SERVERS` remained empty, every application pool remained at zero clients/servers/waiters, the two-address DNS cache stayed unchanged, PgBouncer PID remained 202718, HAProxy master remained 457026, ulc-02 ChirpStack remained `exited|0`, and ulc-01 HTTP remained `200`. Treat F1E-1 as a disposable-client CA harness failure, not as a PgBouncer, HAProxy, PostgreSQL, or resolver failure.

The commissioned PgBouncer CA boundary is already documented and must be used to repair the harness before another connection is attempted: `/etc/lorawan-pki/pgbouncer/ca.crt` is the same internal CA used by PostgreSQL, expected size 1891 bytes and SHA-256

`6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb`, with subject/issuer CN = LoRaWAN PostgreSQL Internal CA. The retry must first verify the host file metadata/hash and OpenSSL parse, then verify the exact file is readable and hashes identically **inside** the disposable Spilo client container. Only after that CA gate passes should exactly one fresh chirpstack client session be opened through ulc-02 PgBouncer. Preserve `pgbouncer.internal.lorawan.com`, `sslmode=verify-full`, and the protected ChirpStack DB password; keep ulc-02 ChirpStack stopped. If the first fresh server connection succeeds, inspect `SHOW SERVERS` and the journal before any `RECONNECT`. Do not batch retries or alter pool sizes; `max_open_connections = 10` remains unchanged.

Phase 9.6G-F8D-F1E-1R repaired the client-side CA harness successfully and produced the first decisive post-repair PgBouncer backend evidence. The commissioned CA validated on the host and inside the disposable Spilo

client at exactly 1891 bytes, SHA-256 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb, and fingerprint

99:00:4B:B3:2D:7D:78:FA:38:61:7C:78:89:6D:7A:7E:FF:9F:A6:10:FC:8F:07:D4:E2:5E:35:25:36:E6:CB:3E. The client itself then received server login has been failing, try again later (server_login_retry) and exited 2, but this is **not** a backend-routing failure: in the same second PgBouncer created a new chirpstack/chirpstack server connection through 127.0.1.1:15432, established TLSv1.3/TLS_AES_256_GCM_SHA384, completed PostgreSQL login, and retained that server as idle with remote_pid=680053. SHOW POOLS simultaneously showed cl_waiting=0, sv_idle=1, and maxwait=0 for the ChirpStack pool. This proves the exact loopback address that failed during F8 now reaches the local HAProxy primary route and completes verified PostgreSQL TLS/authentication successfully.

The remaining server_login_retry client error is a PgBouncer retry-throttle condition, not evidence that the new loopback route failed. PgBouncer is commissioned with server_login_retry = 5; after a backend login/connect failure, new clients arriving during that interval are rejected immediately rather than queued for another connection attempt.

Phase 9.6G-F8D-F1E-1R2 then closed the client-use portion of the loopback repair. SHOW CONFIG confirmed live server_login_retry=5 (default 15, changeable yes). After waiting six seconds, a fresh client session through pgbouncer.internal.lorawan.com:6432 succeeded with client_exit=0 and SQL returned chirpstack|10.104.0.2/32|f|681189, proving the connection reached the current PostgreSQL primary. PgBouncer then showed one idle ChirpStack server at 127.0.1.1:15432, remote_pid=681189, using TLSv1.3/TLS_AES_256_GCM_SHA384; the corresponding journal lines showed a new server connection from 127.0.0.1, successful TLS establishment, and a normal client-close request. SHOW POOLS stayed at cl_waiting=0, sv_idle=1, and maxwait=0, with no new server_login_retry, connect failed, or query_wait_timeout event. ulc-02 ChirpStack remained exited|0, PgBouncer PID remained 202718, HAProxy master remained 457026, and ulc-01 HTTP remained 200. F1E-1R2 is therefore **COMPLETE / PASS** and is authoritative end-to-end proof that the exact formerly failing 127.0.1.1:15432 resolver-selected backend now carries a real PgBouncer client query successfully.

Phase 9.6G-F8D-F1E-2 then issued exactly one RECONNECT chirpstack; with no active ChirpStack application client, re-proved SHOW SERVERS was empty, and created exactly one new verified client session. The new client succeeded with client_exit=0 and SQL returned chirpstack|10.104.0.2/32|f|685960. PgBouncer again selected 127.0.1.1:15432, established TLSv1.3/TLS_AES_256_GCM_SHA384, and retained the connection idle with remote_pid=685960; SHOW POOLS remained cl_waiting=0, sv_idle=1, and maxwait=0, with no connect failed, server_login_retry, or query_wait_timeout in the new journal window. ulc-02 ChirpStack remained exited|0, PgBouncer PID remained 202718, HAProxy master remained 457026, and ulc-01 HTTP remained 200. F1E-2 is therefore **COMPLETE / PASS** for another fresh resolver-selected backend connection after RECONNECT.

Do not issue further RECONNECT commands merely to force address alternation. PgBouncer selecting 127.0.1.1 again does not indicate a fault or guarantee that c-ares alternates addresses on each reconnect. The private 10.104.0.4:15432 path was already proven before the repair to establish PgBouncer backend TLS successfully, and F1C preserved that private listener while adding the loopback listener; direct post-repair TCP checks also proved both addresses open. The defect that caused F8 was specifically that resolver-selected 127.0.1.1:15432 was closed. That exact path is now repeatedly proven end-to-end through PgBouncer, HAProxy, verified PostgreSQL TLS, and a primary SQL query.

Phase 9.6G-F8D-F2 then repeated the original two-node ChirpStack coexistence condition with the routing repair active and all application/PgBouncer pool limits unchanged. ulc-01 started running|restarts=0 with HTTP 200; ulc-02 started from exited|restarts=0, reached HTTP 200 on readiness attempt 2, and both nodes remained running|restarts=0 with HTTP 200 through three 30-second checkpoints. The ANSI-normalized ulc-02 log contained one PostgreSQL setup marker, one integration-client marker, one gateway-client marker, and **zero** ERROR, panic, query_wait_timeout, or connection closed lines. ulc-01 produced zero same-window errors or panics.

PgBouncer evidence during F2 is the decisive repair proof under real ChirpStack load. Three concurrent ChirpStack server connections were created at startup: two through 127.0.1.1:15432 and one through 10.104.0.4:15432. All three established TLSv1.3/TLS_AES_256_GCM_SHA384; SHOW SERVERS reported all three active,

with PostgreSQL remote PIDs 687347, 687348, and 687349. SHOW POOLS showed c1_active=3, c1_waiting=0, sv_active=3, sv_login=0, and maxwait=0 for the ChirpStack pool. The PgBouncer journal contained no connect failed, server_login_retry, or query_wait_timeout. This directly proves **both** addresses in the problematic c-ares cache now work simultaneously through the same local HAProxy primary route. The original F8 failure is therefore resolved without changing max_open_connections=10, default_pool_size=3, max_db_connections=8, query_wait_timeout=15, or any PostgreSQL limit. F2 is **COMPLETE / PASS**, and ulc-02 correctly remains running.

Phase 9.6G-F8D-F3 completed the extended two-node coexistence soak successfully and removes the need for any pool-size change. Starting at 2026-08-26T04:30:44Z, both ChirpStack nodes were already running|0 with HTTP 200 and remained so at four one-minute checkpoints and at the final health check. The same-window ANSI-normalized logs showed ulc01_error_count=0, ulc01_panic_count=0, ulc02_error_count=0, ulc02_panic_count=0, ulc02_query_wait_timeout_count=0, and ulc02_connection_closed_count=0. The ulc-02 PgBouncer journal contained zero connect failed, query_wait_timeout, server_login_retry, server-login-throttle, or TLS-handshake error events during the same four-minute window. F3 is therefore **COMPLETE / PASS**.

PgBouncer remained stable under the real two-node scheduler workload throughout F3. SHOW DNS_HOSTS still exposed the two-address postgres-ha.internal cache. SHOW SERVERS retained three active ChirpStack server sessions that were originally established during F2: two through 127.0.1.1:15432 and one through 10.104.0.4:15432, all using TLSv1.3/TLS_AES_256_GCM_SHA384. SHOW POOLS remained c1_active=3, c1_waiting=0, sv_active=3, sv_login=0, and maxwait=0. The same three PostgreSQL remote PIDs remained active through the extended soak. This is sustained evidence that the existing default_pool_size=3, max_db_connections=8, query_wait_timeout=15, and ChirpStack max_open_connections=10 are adequate for the current few-sensor POC workload; do not raise them preemptively.

Phase 9.6G-F8D-F4 is **PARTIAL / PASSING EVIDENCE PRESERVED; ULC-03 HARNESS RETRY REQUIRED**. The Patroni REST matrix already proved exactly one primary (10.104.0.2 /primary=200, /replica=503) and two replicas (10.104.0.4 and 10.104.0.8 each /primary=503, /replica=200). Direct database checks succeeded on ulc-01 as f|25|30|1.6 with all seven required core tables and on ulc-02 as t|25|30|1.6 with all seven required core tables. The ulc-03 database query and the patronictl --extended topology command did not execute because F4 SSHed from ulc-03 back into 10.104.0.8 and then invoked remote sudo; the existing local sudo -v timestamp is not inherited by that new SSH login, so sudo returned a terminal is required / a password is required. Treat these two omissions as a control-shell harness issue only, not as Patroni/PostgreSQL failure evidence.

All remaining F4 checks passed. Current ulc-01 logs contain exactly one chirpstack-ulc01-integration marker, one chirpstack-ulc01-gateway marker, and two \$share/chirpstack/ subscription markers for the application command and AS923 gateway-event topics. Current ulc-02 logs likewise contain exactly one chirpstack-ulc02-integration, one chirpstack-ulc02-gateway, and the same two shared subscriptions. PgBouncer on both application nodes shows three active ChirpStack sessions, c1_waiting=0, sv_login=0, and maxwait=0; all server sessions use TLSv1.3/TLS_AES_256_GCM_SHA384. ulc-01 routes all three through its private 10.104.0.2:15432; ulc-02 retains two loopback sessions plus one private session, confirming the repaired dual-address path remains healthy. Both ChirpStack containers remain running|0 and both private HTTP endpoints return 200.

Phase 9.6G-F8D-F4R closed the two ulc-03-only omissions locally without SSHing from ulc-03 back into itself. patronictl --extended reported exactly one Leader/running member (ulc-01 at 10.104.0.2) and two Replica/streaming members (ulc-02 at 10.104.0.4 and ulc-03 at 10.104.0.8), all on timeline 3; both replicas showed receive/replay LSN 0/30000000 with lag 0 and no pending restart. Local ulc-03 SQL returned t|25|30|1.6, and the required-core-table count was 7. Together with the already-preserved F4 evidence for the Patroni REST role matrix, ulc-01/02 schema state, four unique ChirpStack MQTT client identities and shared subscriptions, both healthy PgBouncer pools, and both ChirpStack nodes running|0 with HTTP 200, F4/F4R is now **COMPLETE / PASS**.

The PostgreSQL/Patroni/PgBouncer/MQTT infrastructure evidence is therefore closed for this two-node ChirpStack commissioning boundary. Do not repeat resolver, pool-size, migration, or database-schema diagnostics without new failure evidence.

Phase 9.6G-F8D-F5 completed the reciprocal single-application-node survival proof successfully. The pre-stop gate showed both ChirpStack nodes `running|0` with HTTP 200; ulc-02's local Valkey HAProxy path returned `role=master`, and its PgBouncer pool held three active ChirpStack server sessions with `cl_waiting=0`, `sv_login=0`, `maxwait=0`, and TLS 1.3. Only the ulc-01 chirpstack container was then stopped. ulc-01 became `exited|0` and its HTTP endpoint became unavailable as expected, while its commissioned infrastructure listeners `:6432`, `:15432`, `:16379`, and `:18883` remained present, confirming this was an application-instance stop rather than a host or dependency failure.

During the following 90-second survival soak, ulc-02 remained `running|0` with HTTP 200 at all three checkpoints. Its same-window ChirpStack log contained zero `ERROR`, `panic`, `query_wait_timeout`, or `connection closed` lines. PgBouncer remained at three active ChirpStack sessions with zero waiting clients and zero max wait; the repaired dual-address PostgreSQL path stayed live over TLS 1.3. The ulc-02 Valkey HAProxy endpoint still returned `role=master`. Two established TCP sessions remained visible on local MQTT HAProxy `10.104.0.4:18883`, and the current ChirpStack lifecycle still contained both `chirpstack-ulc02-integration` and `chirpstack-ulc02-gateway` connection markers plus both `$share/chirpstack/...` subscriptions. The final gate reported `ULC02_SINGLE_NODE_SURVIVAL=PASS`. F5 is therefore **COMPLETE / PASS**.

Together with the earlier controlled ulc-02 stops that left ulc-01 healthy, Phase 9 now has symmetric proof that stopping either ChirpStack application instance leaves the other functional against its local PostgreSQL/PgBouncer, Valkey, MQTT, and HTTP paths.

Phase 9.6G-F8D-F6 restored only the existing ulc-01 ChirpStack container and completed the final two-node rejoin gate successfully. The pre-start boundary showed ulc-01 `exited|0` while ulc-02 remained `running|0` with HTTP 200. ulc-01 reached HTTP 200 on readiness attempt 2 and both nodes then remained `running|0` with HTTP 200 through three 30-second checkpoints. Fresh ulc-01 logs contained exactly one PostgreSQL setup marker, one Redis setup marker, one `chirpstack-ulc01-integration` marker, one `chirpstack-ulc01-gateway` marker, and two `$share/chirpstack/...` subscription markers, with zero `ERROR`, `panic`, `query_wait_timeout`, or `connection closed` lines. ulc-02 had zero same-window errors, panics, or query-wait timeouts.

The restored ulc-01 PgBouncer pool established three active ChirpStack server sessions through `10.104.0.2:15432`, all using `TLSv1.3/TLS_AES_256_GCM_SHA384`; `cl_waiting=0`, `sv_login=0`, and `maxwait=0`, and the same-window PgBouncer error filter returned no events. The ulc-01 Valkey application path still routed to `role=master`. Two established MQTT TCP sessions were visible through local `10.104.0.2:18883`, and fresh logs showed both unique ulc-01 MQTT client identities plus both expected shared subscriptions. Final state was ulc-01 `running|0` HTTP 200 and ulc-02 `running|0` HTTP 200. `ULC01_REJOIN_GATE=PASS`; F6 is therefore **COMPLETE / PASS**.

Phase 9 is COMPLETE / PASS. The application layer now satisfies the documented acceptance boundary: immutable ChirpStack 4.19.1 image/digest and AS923 configuration are recorded; PostgreSQL migration ownership and final 25-table / 30-Diesel / pg_trgm 1.6 state are proven; both nodes use their local PgBouncer, writable-primary Valkey HAProxy, and redundant ChirpStack MQTT HAProxy paths; both private HTTP endpoints are healthy on `:18080`; pool limits remain adequate without tuning; symmetric single-instance survival is proven; and both nodes have rejoined cleanly with zero application errors. No public `:443`, Reserved IPv4, or public-DNS mutation was required during Phase 9. Handoff is now to Phase 10, which must begin with its documented activation preflight before any public-ingress mutation.

9.11 Container definition - build only after preflight

The final Compose file must be generated from the exact image and resolved dependency paths. The intended shape is:

```
services:
  chirpstack:
    container_name: chirpstack
    image: chirpstack/chirpstack@sha256:9e0105f1dd733d3d3caa77aa7cfdbf817417fab8a093dd89639a2cd899ab9efe
```

```

restart: unless-stopped
command:
  - --config
  - /etc/chirpstack
env_file:
  - /etc/lorawan-cloud/chirpstack/chirpstack.env
volumes:
  - /etc/lorawan-cloud/chirpstack/chirpstack.toml:/etc/chirpstack/chirpstack.toml:ro
  - /etc/lorawan-cloud/chirpstack/region_as923.toml:/etc/chirpstack/region_as923.toml:ro
  - /etc/lorawan-cloud/chirpstack/ca-certificates.crt:/etc/ssl/certs/ca-certificates.crt:ro
extra_hosts:
  - "pgbouncer.internal.lorawan.com:<THIS_APP_PRIVATE_IP>"
  - "valkey.internal.lorawan.com:<THIS_APP_PRIVATE_IP>"
  - "mqtt.internal.lorawan.com:<THIS_APP_PRIVATE_IP>"
ports:
  - "<THIS_APP_PRIVATE_IP>:18080:8080"
stop_grace_period: 45s

```

The container may still listen internally on 8080 if the pinned image uses that default, but **do not bind host port 8080**: it is already owned by the commissioned Spilo/PostgreSQL stack on ulc-01 and ulc-02. Before deployment, choose and prove a free private host port (for example 18080 only if the pre-bind check confirms it is free) and map `<THIS_APP_PRIVATE_IP>:<FREE_CHIRPSTACK_HOST_PORT>:8080`. Do not expose the host port on 0.0.0.0. Do not publish optional gRPC/REST listeners unless a required integration is identified and secured.

9.12 Database migration control

Only one designated ChirpStack node may own an initial or upgrade migration boundary.

Before the first start:

1. verify the existing chirpstack database backup and hash evidence;
2. read the pinned release's migration notes;
3. determine whether startup automatically migrates the schema;
4. nominate ChirpStack-1 on ulc-01 as the first canary unless evidence requires another node;
5. keep ChirpStack-2 stopped during any first migration;
6. capture sanitized migration logs and post-migration schema/application checks;
7. only then start the same digest on ulc-02.

Never allow both nodes to race an undocumented schema migration.

9.13 First deployment order

After sections 9.3-9.12 pass:

1. install the pinned image and reviewed config on **ulc-01 only**;
2. start ChirpStack-1 on the selected free private host port on 10.104.0.2 (container port 8080 only if confirmed by the pinned image);
3. verify database TLS through local PgBouncer;
4. verify Valkey TLS through local :16379;
5. verify MQTT authentication/TLS/topic access through the commissioned MQTT route;
6. verify region loading and a harmless UI/API operation;
7. stop and fix immediately if the canary changes PostgreSQL/Valkey topology or cannot authenticate to MQTT;

8. after the canary passes, install the **same digest and shared configuration hashes** on ulc-02, changing only explicitly node-specific values;
9. start ChirpStack-2 on the same selected private host-port convention on 10.104.0.4;
10. verify both private nodes independently and compare image/config hashes;
11. only after both are healthy, hand public :443 and Reserved-IPv4 work to Phase 10.

Do not require a real gateway or public DNS to prove the first service canary. Real gateway uplink/downlink validation occurs after the cloud application path and public/gateway routing are ready.

9.14 Private health and commissioning checks

Before declaring either ChirpStack node ready, prove all of these from that node's container/network context:

- PgBouncer hostname resolves to the local app host and TLS verifies;
- a harmless PostgreSQL query reaches the current writable Patroni primary;
- `valkey.internal.lorawan.com:16379` verifies TLS and routes to `role:master`;
- the MQTT certificate verifies as `mqtt.internal.lorawan.com` through the commissioned route;
- the ChirpStack MQTT identity is authorized for required topics and denied outside its policy;
- the approved region configuration loads without duplicate/unknown-region errors;
- the selected private ChirpStack host port responds normally and maps to the image's confirmed internal HTTP listener;
- no secret is printed in logs;
- both application nodes use the same image digest and shared file hashes.

If the pinned version has an official read-only readiness endpoint, validate it and document it. Otherwise use the normal private UI/API response plus the separate dependency checks above; do not invent a writable health endpoint.

9.15 Public HTTPS is Phase 10, not this phase

Do **not** add the public HAProxy :443 frontend or move the Reserved IPv4 while the ChirpStack application layer is still being commissioned. Phase 9 finishes with two healthy private ChirpStack nodes.

Phase 10 owns:

```
chirpstack.<DOMAIN>:443 certificate deployment
HAProxy HTTP backend health for the selected private ChirpStack host port on ulc-01/ulc-02
Reserved IPv4 ownership/failover
public DNS validation
public UI/API exposure controls
```

This separation makes rollback clear: a Phase 9 failure cannot disturb the public ingress layer because that layer is not active yet.

9.16 Phase 9 acceptance

Phase 9 is complete only when:

- exact ChirpStack version and immutable digest are recorded;
- exact configuration schema and active region files are recorded and identical where required;

- both nodes use local PgBouncer and verify `pgbouncer.internal.lorawan.com`;
- both nodes use local writable-primary Valkey HAProxy and verify `valkey.internal.lorawan.com`;
- ChirpStack MQTT workload authentication/ACLs are explicitly validated;
- MQTT routing used by the two ChirpStack instances has no single ulc-01 HAProxy dependency;
- one controlled database migration owner is proven;
- ChirpStack-1 and ChirpStack-2 both run privately on the selected non-conflicting host port while the existing Spilo/PostgreSQL :8080 listeners remain untouched;
- both instances use the same shared secrets, image digest, region hashes, and integration policy;
- stopping either ChirpStack instance leaves the other healthy against PostgreSQL, Valkey, and MQTT;
- no public :443 / Reserved-IP change was needed to commission the application layer.

Next phase after these checks pass: [10-self-managed-public-ingress.md](#).

Supporting runbook: 12-gateway-and-device-migration.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

12. Gateway and Device Migration

Status: REQUIRED PRE-TEST SETUP / DRAFT. Public MQTT normal-path activation and the Phase 13A backup safety checkpoint are available. Do not cut over a gateway or device until the remaining Phase 11 Gateway OS normal-path package/configuration is installed and verified on the physical gateway. For the current non-destructive fast path, the verified off-host logical-backup archive/hash is sufficient; isolated restore and destructive DR proof remain deferred to the later Phase 15 boundary. This is a cutover/provisioning phase, not a chaos phase.

This guide supports two real deployment cases: migration from an older authoritative server-hosted ChirpStack installation, or fresh provisioning directly into the new cloud when there is no authoritative legacy database to import. Do not pretend a fresh deployment is a database migration. When a legacy source exists, the source host is the machine that currently runs ChirpStack and its Compose project; it is not the Raspberry Pi Gateway OS appliance.

12.1 Goal

Move the existing ChirpStack state from the **source application/server host** to the cloud without running two authoritative network servers for the same gateway and devices. The Raspberry Pi remains the physical gateway. Preserve tenants, applications, profiles, gateways, devices, root-key state, sessions, frame counters, and integration settings when technically compatible.

12.2 Migration strategy decision

Choose one:

Full ChirpStack database migration

Use when source and target versions are compatible and preserving sessions/frame counters is important. This minimizes device reprovisioning but requires careful version and schema matching.

Recreate configuration and rejoin devices

Use when the source database is incompatible, small enough to rebuild, or security policy requires new root keys/sessions. This creates more field work and must account for battery devices and physical access.

Fresh cloud provisioning

Use when the new cloud is the first authoritative ChirpStack environment. Create the tenant, application, device profile, gateway, and device using the commissioned ChirpStack UI/API; generate/store root keys through the approved secret workflow; register the real Gateway EUI; then perform a normal OTAA join. There is no source

database dump/restore in this branch, but Phase 13A still protects the already-commissioned cloud state before onboarding.

Do not partially copy tables by hand. Use supported database migration, explicit reprovisioning, or the fresh-provisioning branch.

12.3 Source inventory

On the legacy source application host, capture the running services without exposing secrets:

```
cd <SOURCE_COMPOSE_DIR>
docker compose ps
docker compose config --services
docker compose images
git rev-parse HEAD 2>/dev/null || true
sudo ss -lntup
```

Export or capture the source counts and identifiers for users, tenants, applications, device profiles, gateways, devices, multicast groups, API-key references, integrations, regions, codecs, and external alerts or dashboards. These values are compared with the restored target before gateway traffic is allowed.

Keep only key references and protected storage locations, not AppKey/NwkKey values or live API tokens.

12.4 Back up and identify the source state

Create a protected custom-format ChirpStack PostgreSQL dump and configuration archive using the logical-backup controls in [13-backup-restore-and-disaster-recovery.md](#). Back up the physical gateway separately with [Gateway Backup and Recovery](#). Validate the dump catalog, calculate its checksum, and copy both backup sets off their source systems.

Keep the source ChirpStack image/version, PostgreSQL major/version, schema or migration state, active region-file hashes, Gateway EUI, Concentrator channel plan, effective MQTT region prefix, and last successful uplink time. When gateway integrity is enabled, also retain the journal implementation/format versions, latest local sequence/segment hash, latest accepted server checkpoint/receipt, evidence endpoint identity, and any unuploaded closed-segment inventory. These values let the target be compared with the authoritative source without copying live root keys into documentation.

Stop here. Do not cut over if the source backup is unreadable, exists only on the source host, or the target restore has not succeeded in staging.

12.5 Build and validate the cloud before migration

The cloud environment must pass:

- etcd quorum and snapshot test;
- Patroni cluster, replication, and switchover test;
- validated logical dumps of chirpstack and lorawan_telemetry plus the current off-host SHA-256-verified Phase 13A transport archive per [13-backup-restore-and-disaster-recovery.md](#); isolated restore rehearsal is deferred until the later destructive/failover boundary for this POC fast path; WAL/base-backup/PITR evidence is required only if that later production backup profile has actually been enabled;
- PgBouncer pooling and HAProxy primary routing;
- Valkey replication + Sentinel automatic failover test;

- MQTT TLS and per-gateway ACL test using a staging identity;
- when v2 gateway integrity is in scope, evidence-ingest mTLS identity, checkpoint conflict rules, read-only gateway MQTT collector, journal/segment verifier, and trusted-decoder staging tests;
- one ChirpStack node and then two-node application test;
- public TLS and load-balancer health;
- the minimal command-line observability/evidence checks required by this POC.

Use a reserved staging gateway/device or synthetic application event. Do not connect the authoritative production-like gateway yet.

Mandatory Phase 13A boundary: before the cutover/provisioning window, create the protected database/configuration backup set and copy the transport archive off the target Droplets with a matching destination SHA-256. For the current non-destructive fast path, that checkpoint is sufficient; complete the isolated lorawan_telemetry restore rehearsal before Phase 15 destructive/failover testing, not during normal-path commissioning.

Stop here. Do not migrate or provision authoritative device state while any dependency above is being commissioned for the first time. First-time HA debugging and live cutover must not happen in the same window.

12.6 Version compatibility

The safest full migration sequence uses the same ChirpStack application version in source and target for the initial restore. Upgrade only after the cloud copy is stable.

Verify:

- PostgreSQL dump compatibility;
- enabled extensions;
- ChirpStack schema migrations;
- region-file changes;
- codec runtime compatibility;
- MQTT topic prefix;
- Valkey configuration;
- external integration endpoints and credentials.

Do not restore a newer database into an older application image.

12.7 Maintenance window preparation

Define:

```
Start time:
Change owner:
Database owner:
Gateway owner:
Rollback authority:
Expected downtime:
Maximum allowed downtime:
Communication recipients:
Rollback decision deadline:
```

Lower public DNS TTL ahead of the window only when DNS records will change. Confirm local access to the gateway and provider console access to cloud nodes.

12.8 Final cutover sequence

Step 1: Freeze source changes

Prevent administrators from adding/changing devices and integrations. Notify operators that telemetry may be interrupted.

Step 2: Stop source ChirpStack application processing

Stop source ChirpStack application processing and only the server-side gateway ingress components required by the approved cutover. Leave the gateway's selected radio runtime available for endpoint reconfiguration. Do not start a different gateway transport during the migration.

```
cd <SOURCE_COMPOSE_DIR>
docker compose config --services
```

Resolve exact service names and stop only the ChirpStack application and broker components required by the approved plan. This architecture has no server-side Gateway Bridge. Do not stop PostgreSQL before the final dump.

Step 3: Create the final database dump

```
umask 077
docker compose exec -T <SOURCE_POSTGRES_SERVICE> \
  pg_dump -U chirpstack -d chirpstack --format=custom \
  > ~/backups/chirpstack/chirpstack-final-$(date +%Y%m%d-%H%M%S).dump
```

Validate with `pg_restore --list, checksum, and off-host copy.`

Step 4: Restore into the cloud

Restore only into the verified target database and through the approved primary/admin path. Use `--exit-on-error`. Do not restore over a target containing unreviewed production data.

After restore, compare object and business counts with the source inventory.

Step 5: Start one cloud ChirpStack node

Review migration logs. Verify users, tenants, applications, profiles, gateways, devices, integrations, and regions before allowing gateway traffic.

Step 6: Reconfigure Gateway OS MQTT

Follow [11-raspberry-pi-4g-backhaul.md](#).

Keep Gateway OS Base, the Concentrator RAK5146 profile, Gateway ID, approved channel plan, and MQTT Forwarder loopback endpoint unchanged.

MQTT Forwarder remains:

```
server = tcp://127.0.0.1:1883
qos = 1
backend = concentrator
```

Install the cloud CA and unique gateway certificate under `/etc/mosquitto/certs/`, then update the local Mosquitto bridge address and approved topic prefix in `/etc/mosquitto/mosquitto.conf`. Keep UDP Forwarder disabled.

Step 6A: Cut over the gateway evidence endpoint without resetting journal history

Run this step only when the gateway-integrity path is deployed.

Before changing the endpoint:

1. create/confirm the latest accepted source-side checkpoint;
2. upload and verify all closed segments when connectivity permits;
3. record the exact checkpoint receipt, sequence, record hash, and segment hash;
4. install the target evidence CA/client identity through the protected gateway procedure;
5. change only the journal uploader endpoint/identity; do **not** reset sequence, segment ID, or previous hash;
6. make the target verifier import/recognize the approved last anchor under the reviewed migration procedure, or explicitly start a new evidence epoch if governance requires one;
7. send a fresh checkpoint/segment and prove it extends the accepted migration anchor.

Do not run two authoritative checkpoint stores that can independently accept conflicting histories for the same gateway without an explicit replication/migration design.

Step 7: Verify gateway traffic

Observe:

- Gateway OS Concentrator, MQTT Forwarder, local Mosquitto, journal, and uploader health;
- local queue size/drain state plus journal sequence, unuploaded segment backlog, and evidence storage reserve;
- cloud load-balancer TCP connection state, when used;
- broker mutual-TLS identity and exact gateway topics;
- ChirpStack gateway MQTT backend logs and last-seen;
- one real uplink and one safe Class A downlink;
- when enabled, a fresh target-side checkpoint and unique journal -> remote MQTT -> ChirpStack -> trusted-decoder verification result.

Step 8: Verify the already-commissioned two-node ChirpStack/public path

Phase 9 already commissions both ChirpStack instances and Phase 10 commissions both anchor listeners plus manual Reserved-IP mobility. During cutover, re-check both nodes are healthy and verify the current Reserved-IP owner serves the public endpoints. **Do not repeat host-loss or automatic takeover testing here.**

Step 9: Validate integrations and downlinks

Confirm webhook/MQTT/database integrations and one safe Class A downlink. A queued downlink is not proof of device receipt.

12.9 Session and device behavior

A full database migration can preserve sessions and frame counters when versions and data are compatible. Still verify every critical device class.

A device may need OTAA rejoin when:

- sessions were not migrated;
- root keys or JoinEUI changed;

- frame-counter state is inconsistent;
- MAC/region profile changed;
- security policy requires new sessions.

Do not reset frame counters or regenerate root keys as a troubleshooting shortcut. Confirm the vendor-supported rejoin method and expected battery or access impact before the window.

12.10 Prevent dual processing

At no point should the same production gateway publish to two authoritative broker/network-server paths unless a controlled duplicate-handling experiment is explicitly approved.

Before declaring cloud authoritative:

1. verify MQTT Forwarder still points only to the local loopback broker and the local Mosquitto bridge points only to the cloud broker;
2. verify the source application host no longer processes gateway topics;
3. inspect the source Compose services and listeners:

```
cd <SOURCE_COMPOSE_DIR>
docker compose ps
sudo ss -lntup
```

Keep source application containers stopped, not deleted, during the rollback window.

12.11 Rollback triggers

Examples:

- cloud database restore or migrations fail;
- gateway cannot authenticate or remain connected;
- gateway last-seen does not update within the target;
- devices produce MIC/frame-counter/session errors;
- downlinks fail for an approved test device;
- cloud application/database failover is unstable;
- integrations lose or duplicate data beyond the accepted limit.

12.12 Rollback procedure

1. stop both cloud ChirpStack nodes and prevent cloud MQTT command publication;
2. when gateway integrity is enabled, stop accepting new target-side gateway evidence before switching the uploader back, preserve every target checkpoint/segment, and record the last accepted target anchor;
3. restore the previous local Mosquitto bridge endpoint, CA, client certificate, private key, and topic prefix;
4. restore the previous evidence-upload endpoint/identity **without resetting the local journal chain**, using the approved rollback anchor/epoch procedure;
5. restart local Mosquitto without changing MQTT Forwarder's loopback endpoint;
6. start the source broker and ChirpStack services in the documented dependency order;
7. confirm the source database was not modified after the final dump, or reconcile cloud-only changes before rollback;

8. verify source gateway last-seen, one uplink, one safe downlink, and—when enabled—evidence-checkpoint continuity under the rollback procedure;
9. communicate rollback and preserve cloud logs/data/evidence objects for analysis.

Do not run both sides while deciding. Select one authoritative path.

12.13 Source decommissioning

After the agreed observation period and a successful cloud backup/restore drill:

- take one final protected source application backup;
- remove live cloud/database secrets from the source application host;
- remove or disable source ChirpStack, PostgreSQL, Valkey/Redis, MQTT broker, Grafana, and Node-RED containers;
- keep the physical gateway on Gateway OS Base with Concentratord, MQTT Forwarder, local persistent Mosquitto, the reviewed integrity journal/uploader, management networking, and approved monitoring;
- verify the source host and gateway listeners independently;
- update asset inventory and diagrams;
- destroy old backups only after the required retention period, legal holds, rollback window, and tested cloud recovery make them unnecessary.

12.14 Final checks

- Cloud and source versions were reconciled before restore.
- Final source dump and configuration archives are validated and off-host.
- Source services are stopped during production cloud processing.
- Gateway uses a unique TLS identity over 4G.
- Counts, sessions, real uplinks, and safe downlinks are validated.
- When gateway integrity is enabled, the migration preserves an explicit checkpoint boundary/epoch and one fresh target-side lineage verifies before v2 evidence resumes.
- The rollback procedure and protected source/cloud recovery points remain available during the observation window; failure-injection recovery testing is deferred to Phase 15.

Next required setup phase: [12a-node-red-timescale-telemetry.md](#).

Pre-cutover gateway MQTT identity clarification - 2026-08-28

Step 6 below installs the **already-issued and already-server-validated** cloud gateway credential on Gateway OS; it must not be the first time the cloud broker authentication boundary is created. To avoid a circular dependency with Phase 10/11, complete the provider-independent gateway mTLS preparation before the final migration window: commission client-certificate authentication on the gateway-facing cloud :8884 backend, issue the per-gateway `clientAuth` certificate with CN equal to the authoritative EUI, install the exact-EUI ACL, and prove positive/negative authorization through the existing anchor :8883 path using the internal broker identity.

The final public FQDN and Reserved IPv4 are **not required** for that preparation. They are required before Step 6 repoints the physical gateway to the Internet-facing broker. During Step 6, install the previously proven cloud CA/client bundle and change the bridge endpoint only after public `mqtt.<DOMAIN>:8883` validates with the broker certificate's public SAN.

Gateway MQTT cloud client certificate issuance PASS - 2026-08-28

The real physical Gateway EUI 0016c001f139a1cb now has a dedicated cloud MQTT client identity issued on ulc-03 from the commissioned internal CA. Issuance directory: /root/lorawan-pg-ca/gateway-0016c001f139a1cb-issuance-20260828T015149Z; protected transfer bundle is its transfer/ subdirectory. The certificate subject is CN = 0016c001f139a1cb, issuer CN = LoRaWAN PostgreSQL Internal CA, serial D8732205912F3C3AC56E0A01E1E10583, validity 2026-08-28 01:51:51Z through 2027-09-29 01:51:51Z, SHA-256 fingerprint 82:C6:9A:D7:12:5D:8C:45:F3:8F:BA:AB:F9:6E:7E:3B:41:F4:BE:78:95:FE:16:05:FD:58:2E:09:9D:F3:0A:ED, and certificate SHA-256 f348cef6e280dff82722ff908cc96e694faf183eec1dffc55a7abc690cb472d8.

The certificate chains successfully for sslclient, is rejected for sslserver, and its public key matches the RSA-3072 private key. The issuing CA SHA-256 remains 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb; the existing CA serial-file SHA-256 remained byte-identical at 50df8c462ef9465ab9198284fa1234f0cbfa4f33eb9779ce6d50dd23a618463d because issuance used an explicit random serial. GATEWAY_MQTT_CLIENT_CERT_ISSUANCE=PASS. No Mosquitto, HAProxy, DNS, firewall, or gateway runtime was changed by issuance.

Next boundary: canary-harden ulc-01 gateway-facing Mosquitto :8884 only to require client certificates, map certificate CN to MQTT username, and enforce an exact-EUI as923 gateway ACL. Preserve ChirpStack :8885 and Node-RED :8886. After ulc-01 passes direct mTLS verification with the new gateway certificate, repeat the same bounded rollout on ulc-02.

ulc-01 gateway mTLS canary PASS - 2026-08-28

ulc-01:8884 was hardened from server-TLS-only to gateway client-certificate authentication without changing the ChirpStack :8885 listener. The active gateway listener now has require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/gateway.acl. The exact Gateway EUI 0016c001f139a1cb is permitted to write only its own as923/.../event/# and state/# topics and read only its own command/# hierarchy. Mosquitto restarted successfully, :8884 and :8885 remained listening, the issued gateway certificate completed TLS and MQTT CONNECT/SUBSCRIBE successfully, and a no-client-certificate connection was rejected. Rollback copy: /etc/mosquitto/gateway-mtls-20260828T015613Z. Next boundary: apply the same bounded change to ulc-02 and prove the same certificate/ACL behavior there before transferring the cloud certificate bundle to Gateway OS.

ulc-02 gateway mTLS rollout PASS - 2026-08-28

ulc-02:8884 now matches the proven ulc-01 gateway-authentication boundary: require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/gateway.acl. Gateway EUI 0016c001f139a1cb is limited to its own AS923 event/state/command hierarchy. Mosquitto restarted successfully; :8884 and ChirpStack :8885 remained listening. The issued gateway certificate completed TLS, MQTT CONNECT, and own-command SUBSCRIBE successfully, while a no-client-certificate connection was rejected. Rollback copy: /etc/mosquitto/gateway-mtls-20260828T015801Z. Therefore both cloud MQTT broker backends now enforce the intended per-gateway mTLS identity. Next boundary: transfer only the three-file cloud certificate bundle (ca.crt, 0016c001f139a1cb.crt, 0016c001f139a1cb.key) to Gateway OS; do not change the working local Mosquitto topology yet.

ulc-02 gateway mTLS state observed - 2026-08-28

Live read-only inspection after the first rollout wrapper showed ulc-02 is already in the intended gateway mTLS state: /etc/mosquitto/conf.d/tls.conf has listener 8884, require_certificate true, allow_anonymous false, use_identity_as_username true, and acl_file /etc/mosquitto/gateway.acl; /etc/mosquitto/gateway.acl exists as 0640 root:mosquitto; per_listener_settings true remains active; and the dedicated ChirpStack listener remains 10.104.0.4:8885 with its own password/ACL files. Therefore do not reapply the mutation. The remaining boundary is a direct positive mTLS/MQTT proof with gateway certificate CN 0016c001f139a1cb plus a no-client-certificate rejection proof.

Both cloud gateway MQTT brokers mTLS acceptance PASS - 2026-08-28

Direct verification from u1c-03 against u1c-02:8884 using the issued gateway identity CN = 0016c001f139a1cb passed TLS, MQTT CONNECT, and subscription to the gateway's own as923/gateway/0016c001f139a1cb/command/# hierarchy. A client without a certificate was rejected. Combined with the earlier u1c-01 canary proof, both gateway-facing Mosquitto backends now enforce per-gateway mTLS and the exact EUI ACL while preserving the dedicated ChirpStack :8885 listeners. The cloud broker-side gateway authentication boundary is therefore complete. Next: transfer only ca.crt, 0016c001f139a1cb.crt, and 0016c001f139a1cb.key from the protected u1c-03 issuance bundle to Gateway OS; do not alter the local loopback broker topology during certificate import.

Streamlined Phase 12 entry gate - 2026-08-28

The Phase 13A fast backup checkpoint is now PASS: phase13a-20260827T032756Z.tar.gz exists off-host on the Windows administration workstation and its SHA-256 matches the u1c-03 source (e97d50c31252ede1fe55b734b6686f270e92ebecb69a36d637b04fbf726cda1c). Do not repeat database dumps or restore rehearsal during normal-path commissioning.

Phase 12 is no longer blocked by the public MQTT endpoint. smartagri-mqtt.duckdns.org:8883 through Reserved IPv4 129.212.208.168 has passed public-name verification, real gateway mTLS CONNECT, and authorized SUBSCRIBE. The gateway-side cloud certificate, exact-EUI ACL, local Mosquitto buffer, SIM7600 QMI data session, and public LTE connectivity are also proven. The remaining cutover boundary is physical Gateway OS access: repoint the bridge, preserve strict TLS/mTLS, prove QoS 1 delivery and application flow, then promote LTE according to Phase 11. Reserved-IP automatic failover remains a separate HA test, not a normal-path blocker.

Production ChirpStack gateway/device provisioning PASS - 2026-09-01

The cloud ChirpStack 4.19 registry was verified empty before provisioning, then populated through the supported authenticated gRPC API rather than by copying rows from the older ChirpStack 4.9 lab database. Current production objects are:

```
application:    dissertation-sensors
gateway:        Gateway-01
gateway EUI:    0016c001f139a1cb
device profile: EMU-01 RAK4631 AS923
device:         dissertation-emu-01
device EUI:     ac1f09fffe296d29
JoinEUI:        0000000000000000
region:         AS923 / region id as923
LoRaWAN MAC:    1.0.2
regional params: B
activation:      OTAA
class:          A
ADR algorithm:   default
expected production normal uplink: nominal 300 seconds with +/-15-second jitter (285-315 seconds); 15-second timing is reserved for the dissertation counted-test profile only
```

The EMU-01 LoRaWAN root key was transferred through protected temporary files and verified byte-for-byte against the previously working lab key without recording or printing the key value. The production JavaScript payload codec was likewise verified byte-for-byte against the reviewed payload-v2 decoder. SEC-02 remains intentionally outside the permanent production registry as a security/test fixture. The temporary global ChirpStack provisioning API key was revoked after provisioning and its protected on-host token file was removed. PRODUCTION_CHIRPSTACK_REGISTRY=PASS is authoritative; tomorrow's remaining acceptance requires the real flashed gateway and EMU-01 radio uplink, not more server object creation.

Supporting runbook: 17-troubleshooting.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

17. Cloud Troubleshooting

Status: STANDBY / LIVING DRAFT. Keep only troubleshooting that matches deployed components. The etcd/network lessons can be retained now; future service sections must be refined from real failure evidence as those technologies are introduced.

Start at the lowest layer that does not show a healthy result. Do not restart every node or rotate credentials until the failing boundary is identified.

```
RAK5146
-> Concentratord
|
+-> delivery: MQTT Forwarder
|   -> gateway-local Mosquitto listener/persistence
|   -> DNS / 4G / MQTT mutual-TLS bridge
|   -> Reserved IPv4 -> current HAProxy public-ingress owner
|       |-> read-only gateway MQTT evidence collector
|       +-> ChirpStack MQTT region backend
|
+-> evidence: integrity journal
|   -> local sequence/hash/segments
|   -> DNS / 4G / HTTPS-mTLS evidence upload
|   -> checkpoint/segment store + verifier

ChirpStack -> PostgreSQL / Valkey -> application integrations
Node-RED -> lorawan_telemetry [TimescaleDB hypertables] -> trusted decoder comparison -> v2 evidence gate
lorawan_telemetry.fabric_outbox -> Fabric adapter-1/2 -> OpenBao-1/2/3 -> external Fabric Gateway
```

Gateway commands in Sections 16.1–16.3 run over SSH on the Raspberry Pi. Cloud commands run on the host that owns the named service. Use one reserved gateway and device event to compare timestamps across layers.

17.1 Radio or Gateway ID failure

Failing layer: RAK5146 hardware access or Concentratord configuration.

Run on the gateway:

```
monit status
uci show chirpstack-concentratord
logread -e chirpstack-concentratord
```

Healthy output shows one running Concentratord instance, the RAK5146/SX1302-or-SX1303 profile, the confirmed legal channel plan, and a stable 16-hexadecimal Gateway ID. SPI, reset, calibration, repeated restart, missing-ID, or changing-ID messages point to profile, HAT seating, power, frequency variant, or hardware trouble.

Correct only the first observed mismatch. Power down before reseating hardware, and stop RF transmission when the region is wrong. Verify by rebooting and confirming the same Gateway ID and channel plan without error loops.

17.2 MQTT Forwarder cannot publish locally

Failing layer: MQTT Forwarder to the gateway-local Mosquitto listener.

Run on the gateway:

```
uci show chirpstack-mqtt-forwarder
logread -e chirpstack-mqtt-forwarder
logread -e mosquitto
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

Healthy configuration uses `tcp://127.0.0.1:1883`, QoS 1, Protobuf, backend `concentratord`, and a listener bound only to loopback. Connection refusal means Mosquitto is stopped or invalid; a WAN address means the buffer is bypassed; a non-loopback listener means the broker is exposed.

Validate the broker configuration, correct the differing address, QoS, backend, or listener, and restart only the affected process. Verify with one real gateway event at the local broker.

17.3 The local queue grows and does not drain

Failing layer: gateway backhaul, DNS, time, TLS, broker ACL, remote MQTT availability, or queue capacity.

Run on the gateway:

```
logread -e mosquitto
nslookup mqtt.<DOMAIN>
ip route
date -u
df -h /etc/mosquitto/data
ls -l /etc/mosquitto/data/mosquitto.db
```

Symptom	Meaning	Smallest safe action
DNS failure	Resolver, APN, or 4G route problem	Restore DNS or route and keep the queue running
Hostname or CA error	Wrong trust file or certificate SAN	Install the intended CA or endpoint; never disable verification
Client rejected	Certificate, expiry, EKU, CN, or key mismatch	Compare the certificate identity and private-key match
ACL denial	Gateway identity or regional topic mismatch	Compare Gateway EUI, certificate CN, and exact topics
Queue grows with a connected bridge	Remote drain rate is below incoming rate	Check broker health and measured publish rate
Queue reaches its finite limit	Outage exceeded the buffer design	Restore the remote path, protect free space, and identify the affected interval
Database missing after reboot	Volatile path or persistence failure	Stop claiming durable buffering and repair the storage path

Healthy recovery reconnects both bridge clients, preserves the queue database, drains old events, and continues accepting new uplinks. Verify at the remote broker and then at ChirpStack; do not delete the queue to make disk usage fall.

17.3A Journal does not advance while MQTT is healthy

Failing layer: independent Concentrator evidence path.

Check the reviewed journal service, its source-interface configuration, current sequence/open segment, persistent evidence path, permissions, and free space using the implementation's approved diagnostics.

Interpretation:

```
Concentrator + MQTT healthy, journal process down
-> journal service/implementation fault

journal process healthy, sequence not advancing for known uplink
-> Concentrator event subscription/filter/source-contract fault

sequence advances, writes fail
-> journal storage/permission/capacity fault
```

Do not point the journal at Mosquitto as a shortcut. Preserve the first affected time/sequence and classify unrecoverable source evidence as `evidence_gap`.

17.3B MQTT is healthy but cloud checkpoint is stale

Failing layer: evidence uploader, DNS/4G route to `evidence.<DOMAIN>`, mTLS identity, ingest API, or evidence-store write path.

Check independently:

```
DNS/route/time to evidence endpoint
server certificate SAN/CA
per-gateway client identity/expiry
last local sequence/closed segment
unuploaded segment count/bytes
latest accepted server checkpoint age
API rejection/conflict logs
```

Healthy MQTT does not clear this incident. Restore upload and prove the next accepted segment/checkpoint extends the previous server anchor.

17.3C Journal hash, checkpoint, or journal-to-MQTT conflict

Failing layer: evidence integrity/correlation boundary.

Freeze v2 evidence promotion for the affected gateway. Preserve the journal object, accepted checkpoint/receipt, captured remote gateway MQTT object, PHYPayload/source digests, software versions, and relevant logs.

Do not renumber records, recalculate a replacement local chain, delete the newer checkpoint, or ask Node-RED which payload to trust.

Use:

```
missing required evidence with no contradictory bytes -> evidence_gap
proven hash/payload/rollback conflict                  -> integrity_failure
```

17.4 The remote broker receives events but ChirpStack does not

Failing layer: remote MQTT authorization or the ChirpStack regional MQTT backend.

On the MQTT host, confirm the gateway event topic and certificate identity. On each ChirpStack application node, inspect the service logs and active region configuration. The regional topic prefix, Protobuf backend, broker endpoint, client certificate, and trust root must agree.

Healthy behavior updates the registered Gateway EUI's last-seen time after a fresh event. If the broker has the message and ChirpStack does not, correct only the differing regional backend, topic prefix, ACL, or credential and restart the affected ChirpStack node. This architecture does not require ChirpStack Gateway Bridge.

17.5 Duplicate application or telemetry rows

Failing layer: application idempotency after valid at-least-once MQTT delivery.

Verify the Node-RED stable event key, ChirpStack `deduplicationId` use, Timescale hypertable status, and the `lorawan_telemetry` uniqueness indexes. Replay one sanitized event. A healthy result leaves one uplink and one row per metric. More rows mean the event key or database constraint is wrong. Fix that layer; do not lower gateway QoS.

If inserts fail only after a Patroni promotion, first verify the promoted node has the compatible TimescaleDB library/preload configuration and that `SELECT extversion FROM pg_extension WHERE extname='timescaledb';` succeeds. A PostgreSQL replica is not a valid telemetry primary if its TimescaleDB runtime is missing or mismatched.

17.5A Telemetry row exists but v2 verification fails

Failing layer: application evidence comparison, not necessarily Node-RED ingestion availability.

Inspect the matching `gateway_evidence.event_verification` result and reason. If `journal/MQTT/ChirpStack` lineage is valid but the trusted decoder disagrees with the row stored in `lorawan_telemetry`, preserve:

```
accepted raw ChirpStack application-data digest
trusted decoder ID/version or code hash
trusted normalized digest/value
Node-RED decoder version
stored telemetry row identity/value
verification ID/reason
```

Do not change the database value merely to make the evidence green. Determine whether the trusted decoder, Node-RED mapping, source bytes, or correlation policy is wrong, then correct the implementation and process a new governed result according to the incident procedure.

17.6 A stale downlink appears after reconnect

Failing layer: downlink session cleanup or an automation publisher.

Confirm the gateway's `cloud-downlink` bridge uses `cleansession true` and `command topic in 0`. Check the remote broker for retained command messages and duplicate client sessions. Healthy behavior does not replay a Class A command after its receive window.

Disable automatic downlinks until the retaining session or publisher is removed, then verify with one harmless, observable command.

17.7 Reserved-IP / MQTT public failover does not preserve service

Failing layer: Reserved-IP ownership, failover agent, etcd lock, DigitalOcean API, HAProxy routing, backend health, timeout behavior, or broker session design.

First check the current public owner from an authenticated admin context:

```
doctl compute reserved-ip get <RESERVED_IP> --format IP,Region,DropletID,DropletName --no-header
```

Then on ha-01/02 check:

```
systemctl status lorawan-public-ingress.timer --no-pager
systemctl status lorawan-public-ingress.service --no-pager -l
journalctl -u lorawan-public-ingress.service --since=-15min --no-pager
sudo /usr/local/sbin/lorawan-ingress-health local
sudo /usr/local/sbin/lorawan-ingress-health public
```

If etcd quorum is unavailable, automatic Reserved-IP movement is intentionally blocked. If the DigitalOcean API/token cannot reassign the address, internal HA can remain healthy while the public endpoint stays on the failed owner.

For MQTT itself, HAProxy owns public anchor :8883, Mosquitto-1/2 listen on private TLS backend :8884, and HAProxy keeps MQTT TLS pass-through so the broker receives the gateway client certificate. A healthy recovery reconnects the gateway and resumes queue drain without changing DNS, certificate, or topics.

Check the intended listeners first:

```
# ha-01 / ha-02 HAProxy
sudo ss -lntp | grep -E ':(8883|18883)\b'

# Mosquitto backend host
sudo ss -lntp | grep ':8884'
```

If HAProxy and Mosquitto are both trying to bind the same VPC :8883, the old port-collision design is still active. Public HAProxy must bind the **anchor IP** on :8883; Mosquitto stays on private backend :8884.

Reserved-IP/HAProxy failover does not synchronize independent Mosquitto sessions, retained messages, or queues. Broker service failover still depends on the gateway's bounded local QoS 1 queue and reconnection behavior.

17.8 UDP Forwarder traffic appears

Failing layer: an unsupported gateway backend is enabled.

Run on the gateway:

```
uci show chirpstack-udp-forwarder
monit status
```

Healthy output has no UDP server and the service disabled. Remove the unintended entries, disable only the UDP Forwarder, and verify the MQTT path with a real uplink.

17.9 ChirpStack reports database or state errors

Failing layer: ChirpStack, PgBouncer, HAProxy, PostgreSQL/Patroni, or Valkey/Sentinel.

Check in this order.

Step 1 - PostgreSQL/Patroni role

```
patronictl -c <PATRONI_CONFIG> list <PG_SCOPE> --extended
```

Expected: exactly one primary and two replicas in the normal state.

Check each Patroni role endpoint from the affected HAProxy host:

```
for h in <HA01_PRIVATE_IP> <HA02_PRIVATE_IP> <HA03_PRIVATE_IP>; do
  printf '%s ' "$h"
  curl -sS -o /dev/null -w '%{http_code}\n' "http://$h:8008/primary"
done
```

Exactly one should be the primary according to the pinned Patroni endpoint semantics. Adapt the command for REST TLS/auth when enabled.

Step 2 - HAProxy DB route

```
psql 'host=postgres-ha.internal hostaddr=<THIS_HOST_PRIVATE_IP> port=15432 dbname=postgres user=
<MONITOR_ROLE> sslmode=verify-full' \
-c 'SELECT inet_server_addr(), pg_is_in_recovery();'
```

Expected: pg_is_in_recovery = false.

Step 3 - PgBouncer

```
psql '<PGBOUNCER_ADMIN_DSN>' -c 'SHOW POOLS;'
psql '<PGBOUNCER_ADMIN_DSN>' -c 'SHOW SERVERS;'
```

Look for sustained `cl_waiting`, repeated login failures, stale server connections after a promotion, or a pool using an unexpected database/host.

Step 4 - application DB route

From the affected application host, query through its normal `:6432` endpoint. Do not bypass PgBouncer to make the test pass.

An authentication error calls for checking the exact role/verifier/secret reference. Connection exhaustion calls for checking pool and PostgreSQL limits. A read-only/no-primary error calls for Patroni and HAProxy diagnosis.

Restarting all application/database nodes together destroys evidence and can remove quorum. Correct the first failing dependency, then verify one harmless API/SQL operation and one fresh uplink.

17.9A etcd quorum is unhealthy

Failing layer: Patroni coordination/DCS.

From an approved administrative shell, first use the transport that is actually deployed. The current cluster uses HTTP on 10.104.0.0/20:

```
ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health

ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  member list --write-out=table
```

If etcd TLS has been deployed since this baseline, use the matching tested CA/client credentials instead. A protocol mismatch (https:// against the current HTTP listener, or the reverse) can look like a service failure.

Interpretation:

```
3/3 reachable -> normal
2/3 reachable -> degraded but quorum can progress; restore the missing member before another failure
1/3 reachable -> no safe majority; stop failover testing and recover quorum
```

Do not remove/re-add multiple members at once. Preserve the member IDs and the pre-test snapshot before destructive membership repair.

17.9B Fabric outbox grows while telemetry is healthy

Failing layer: Fabric adapter worker, OpenBao KMS, external Fabric Gateway, or commit reconciliation.

Check in this order:

```
fabric_outbox status counts plus finalized_payload eligibility
for v2, matching verifier-owned status='verified'
ULC-01 adapter-1 process/database connectivity and enabled ownership
ULC-02 adapter-2 remains write-disabled unless an approved takeover is in progress
live/expired job leases
reconciling rows and durable txid/record_id/prepared-transaction/commit-status material
legacy submitted_unknown rows, if any, handled as compatibility reconciliation only
needs_attention conflicts
openbao-kms.internal.<DOMAIN>:18200 TLS reachability
OpenBao 3-member state, seal state, and quorum
Transit sign/verify errors
HRC private Fabric Gateway TLS/MSP/channel/chaincode reachability and peer commit-status visibility
```

Do not make Node-RED call Fabric synchronously to reduce the queue. Restore the first failing asynchronous dependency and let the durable outbox drain under the normal retry/reconciliation rules.

17.9C OpenBao is sealed, degraded, or has no quorum

Failing layer: evidence KMS.

One unavailable OpenBao member should leave 2/3 quorum. Two unavailable voters or sealed members can stop new Raft writes/signing. Preserve the PostgreSQL outbox/telemetry, restore the missing/unsealed cluster members using the documented recovery material, and verify the stable KMS endpoint before restarting adapter retries.

Use the pinned CLI through the stable service name and, when needed, directly against members:

```
bao status
bao operator raft list-peers
```

Also test the HAProxy health target directly with HTTPS and the internal CA:

```
curl --fail --silent --show-error \  
--cacert /etc/lorawan-pki/openbao/ca.crt \  
'https://openbao-kms.internal.<DOMAIN>:18200/v1/sys/health?standbyok=true'
```

If HAProxy reports a sealed/uninitialized node healthy, its KMS health check has regressed to a TCP-only check or is using the wrong health semantics. Repair that before adapter retries.

Never export/create a replacement local signing key in the Fabric adapter to bypass OpenBao. If the historical Transit key cannot be recovered, stop attestation recovery and preserve the affected sealed/unsealed evidence for governance review.

17.9D Valkey/Sentinel does not fail over or HAProxy routes a replica

Failing layer: Valkey replication, Sentinel quorum/authentication, or HAProxy primary-role health check.

Check each Valkey node with TLS and the protected application/admin credential:

```
valkey-cli --tls --cacert <VALKEY_CA> -h <VALKEY_IP> -p 6379 \  
-a '<LOAD_PROTECTED_SECRET>' ROLE
```

Check every Sentinel:

```
valkey-cli --tls --cacert <VALKEY_CA> -h <SENTINEL_IP> -p 26379 \  
-a '<LOAD_SENTINEL_SECRET>' SENTINEL CKQUORUM lorawan-valkey  
  
valkey-cli --tls --cacert <VALKEY_CA> -h <SENTINEL_IP> -p 26379 \  
-a '<LOAD_SENTINEL_SECRET>' SENTINEL GET-MASTER-ADDR-BY-NAME lorawan-valkey
```

Then query the app endpoint:

```
valkey-cli --tls --cacert <VALKEY_CA> \  
--sni valkey.internal.lorawan.com \  
-h <LOCAL_HAPROXY_PRIVATE_IP> -p 16379 \  
-a '<LOAD_PROTECTED_SECRET>' ROLE
```

Expected: the HAProxy endpoint reports primary/master. If direct Sentinel state is correct but HAProxy points at a replica, inspect the commissioned HAProxy TLS health sequence: dedicated haproxy-health authentication, exact CRLF INFO replication, and min-recv 64 string role:master. A simple PING is insufficient because replicas also answer PING.

Restore 3 data nodes + 3 Sentinels and a passing CKQUORUM before another failure test.

17.10 An upgrade causes a regression

Preserve the Gateway OS image, local broker data/configuration, journal implementation/configuration and continuity metadata, certificates, latest accepted checkpoint/evidence objects, cloud image digests, database versions, active schema state, and logs from the failed attempt.

Restore the previously tested component without deleting data volumes or rewinding the authoritative checkpoint store. Verify the rollback in layers: gateway radio, local publish, journal source/sequence/hash, WAN queue, evidence upload/checkpoint continuity, mutual TLS, remote broker, gateway-MQTT evidence capture, ChirpStack gateway activity, OTAA/uplink, trusted decoder comparison, database state, and integrations. A process that merely starts is not a successful rollback.

Canonical source: [deployment/technology-guides/05-chirpstack.md](#). For any mutable production-status statement, check the dated baseline and verified live state.